



貴州大學

本科毕业论文（设计）

论文（设计）题目：基于流量字节短序列的远控木马早期检测程序设计与实现

学 院：计算机科学与技术学院（贵州保密学院）

专 业：计算机科学与技术

班 级：计科 2101

学 号：2100170268

学生姓名：苏洁

指导教师：郭春

2025 年 6 月 09 日

贵州大学本科毕业论文（设计）

诚信责任书

本人郑重声明：本人所呈交的毕业论文（设计），是在导师的指导下独立进行研究所完成。毕业论文（设计）中凡引用他人已经发表或未发表的成果、数据、观点等，均已明确注明出处。

特此声明。

设计作者签名： 朱洁

日 期： 2025.6.09



目录

摘要	IV
ABSTRACT	V
第一章 绪论	1
1.1 研究背景及意义	1
1.2 远控木马检测技术国内外发展现状	2
1.2.1 国内外发展现状	2
1.2.2 发展趋势	5
1.3 主要研究内容	6
1.4 论文组织安排	6
第二章 相关技术	8
2.1 远控木马概述	8
2.1.1 远控木马的定义	8
2.1.2 远控木马特性	8
2.1.3 远控木马通信行为	9
2.2 网络流量分析技术	10
2.2.1 流量采集方式与工具	10
2.2.2 网络流量的表示方式	11
2.3 特征提取技术	11
2.3.1 n-gram 模型介绍	11
2.3.2 TF-IDF 模型介绍	12
2.4 机器学习	12
2.4.1 机器学习的概念和模型	12



2.4.2 随机森林介绍	12
2.5 本章小结	13
第三章 基于流量字节短序列的远控木马早期检测方法	14
3.1 远控木马流量序列分析	14
3.2 基于流量字节短序列的远控木马早期检测方法	15
3.2.1 流量预处理	16
3.2.2 特征计算	17
3.2.3 模型训练与检测	18
3.3 实验结果与分析	18
3.3.1 实验环境与数据集	18
3.3.2 评估指标	21
3.3.3 实验结果与分析	21
3.4 本章小结	24
第四章 基于流量字节短序列的远控木马早期检测系统分析与设计	25
4.1 系统需求分析	25
4.1.1 功能需求	25
4.1.2 非功能需求	26
4.2 系统总体设计	26
4.2.1 设计原则	26
4.2.2 设计目标	27
4.2.3 系统工作流程	27
4.3 系统功能设计	28
4.3.1 系统功能模块划分	28
4.3.2 系统功能模块设计	28



4.4 本章小结	33
第五章 系统实现	34
5.1 系统部署方式	34
5.2 实时网络流量捕获模块实现	34
5.3 远控木马流量检测与自动终止模块实现	34
5.4 邮件告警模块实现	38
5.5 Pcap 包离线检测模块实现.....	39
5.6 检测报告生成模块实现	39
5.7 本章小结	40
第六章 系统测试	41
6.1 系统测试环境	41
6.2 功能测试	42
6.2.1 实时网络流量捕获.....	42
6.2.2 远控木马流量检测与自动终止.....	43
6.2.3 邮件告警.....	45
6.2.4 Pcap 包离线检测	45
6.2.5 检测报告生成.....	47
6.3 测试总结	48
6.4 本章小结	48
第七章 总结与展望	49
7.1 总结	49
7.2 展望	49
参考文献.....	50
致谢	52



基于流量字节短序列的远控木马早期检测程序设计与实现

摘要

远程控制木马（以下简称“远控木马”）是一种通过网络连接为攻击者提供远程控制权限的恶意软件，具有高度隐蔽性，一旦感染目标主机可能导致敏感信息泄露并严重威胁数据安全。国内外研究人员近年来针对远控木马检测和防御进行了一系列研究，但这些技术方案存在着对未知远控木马的检测准确率较低或及时性不足、资源消耗较高的问题。

为解决上述问题，本文设计了一种基于流量字节短序列的远控木马早期检测方法，并设计实现了相应的检测系统。该方法聚焦于远控木马在其运行初始阶段的客户端通信行为，从客户端建立连接后发出的前两个有载荷的 TCP 数据包中提取字节序列，利用随机森林算法完成检测。相比于依赖长时间流量或深度学习的方法，该方案具备轻量级、高效的优点。实验结果表明，该方法在检测未知远控木马流量时准确率可达 97.3%，泛化性强。在此基础上，本文基于 PySide2 框架构建了其原型系统，将流量监控、远控木马流量检测和自动终止措施集成于一体。实现了远控木马检测与响应的完整闭环，具备良好的实用性与部署前景。

关键词： 远程控制木马, 网络通信行为, 字节序列分析, 机器学习, 早期检测



Design and Implementation of an Early Detection Program for Remote Access Trojan Based on Short Byte Sequences Extracted from Network Traffic

ABSTRACT

Remote Access Trojan (RAT) is a type of malware that grant attackers remote control capabilities over target systems via network connections. Characterized by high stealth, RATs can cause sensitive information leakage and pose serious threats to data security once infiltrated. In recent years, researchers at home and abroad have conducted a series of studies on the detection and defense of RATs. However, existing technical solutions still face challenges in accurately or promptly detecting unknown RATs, or high computational overhead.

To address the above issues, this paper proposes an early detection method for RATs based on short byte sequences extracted from network traffic, and further designs and implements a corresponding detection system. The proposed method focuses on the client's communication behavior during the early stage of RAT activity, extracting byte sequences from the first two payload- carrying TCP packets sent by the client after connection establishment, and employs machine learning techniques for detection. Compared with methods that rely on long-term traffic analysis or deep learning, this approach offers the advantages of being lightweight and efficient. Experimental results show that the proposed method achieves a detection accuracy of up to 97.3% for unknown RAT traffic. On this basis, a prototype system was developed using the PySide2 framework, integrating traffic monitoring, RAT detection, and automatic termination mechanisms. The system realizes a complete closed-loop for RAT detection and response, demonstrating strong practicality and deployment potential.

Keywords: Remote Access Trojan, Network Communication Behavior, Byte Sequence Analysis, Machine Learning, Early Detection



第一章 绪论

1.1 研究背景及意义

远控木马(Remote Access Trojan, RAT)是一种恶意软件,它采用 Client/Server 架构,植入目标主机能够对其实施全面控制以窃取敏感数据、执行恶意操作等,严重威胁着个人隐私安全,还可能导致企业敏感信息泄露,甚至对国家的网络安全造成重大危害。根据国家互联网应急中心(CNCERT)发布的《2021 年上半年我国互联网网络安全监测数据分析报告》^[1]中的结果显示,2021 年上半年我国境内受恶意程序攻击的 IP 地址近 3048 万个,约占我国 IP 地址总数的 7.8%。其中,远程控制类恶意程序的增幅尤为显著,与前一年相比增加了 50%。此外,根据中国网络空间安全协会与国家互联网应急中心等单位联合发布的《2023 年网络安全态势研判分析年度综合报告》^[2],深信服千里目安全中心全年共拦截恶意程序 210.3 亿次,其中木马远控程序占比 27.64%,位居第二大类。如图 1.1 所示,根据瑞星公司发布的《2024 年中国网络安全报告》显示^[3],2024 年瑞星“星核”平台共截获病毒样本总量 6848 万个,病毒感染次数 8474 万次,报告期内,新增木马病毒 4552 万个,为第一大种类病毒。可以看出,远控木马作为一种主要的恶意程序,其快速增长和广泛传播对信息安全构成严重威胁,检测和防御远控木马已然成为保障我国网络空间安全的关键举措,对维护国家安全、经济发展和社会稳定具有重要意义。

鉴于远控木马对网络空间构成的严峻挑战,国内外研究者提出了多种检测方法。主要可分为基于静态文件的检测方法、基于行为分析的检测方法和基于网络流量的检测方法。1) 基于静态文件的检测方法通过从应用程序的相关静态文件中提取操作码序列、PE 头信息等静态特征来实现木马检测,然而,随着远控木马越来越多地采用加壳技术和代码混淆^[4],其文件或结构将产生显著变化,导致其检测准确率明显下降,泛化能力较弱^[5]。2) 基于行为分析的检测方法通过在虚拟化环境中运行程序,监控程序运行时的行为和系统的变化来实现检测,该方法能够有效检测加壳和代码混淆的木马^[6],但其由于需要在虚拟化环境中运行目标程序,对资源的消耗较高且耗时长,难以实现快速检测,同时许多远控木马集成了虚拟机检测技术^[7-9],它们通过在运行前检查执行环境以避免在沙箱或虚拟化环境中被分析和检测。3) 基于网络流量的检测方法通过分析程序服务端和客户端之间交互的流量来实现远控木马的检测,是一种当前主流的检测方法,

具体可分为基于统计特征的方法和基于字节序列的方法，前者依赖于流量的统计特征（如上下行数据包大小等）的提取，往往需要长时间收集网络流量而导致检测及时率不佳；后者通常使用深度学习模型，存在计算成本高的问题，且常为适应模型输入长度限制导致序列信息截断，导致信息丢失影响检测效果。此外，深度学习的效果通常依赖于大规模且多样化的训练数据，当面对训练集中未充分覆盖的攻击样本或应用行为时，其检测性能可能显著下降，表现出较弱的泛化能力^[10]。因此，如何准确且高效地检测远控木马仍然是一个待解决的问题，该问题的解决对及时检测远控木马、避免受害主机遭受重大损失具有重大意义。



图 1.1 2024 年病毒类型统计

1.2 远控木马检测技术国内外发展现状

1.2.1 国内外发展现状

随着互联网的快速发展，远控木马的传播途径愈加多样化，其攻击技术也不断进步。为了躲避传统的安全防护措施，现代远控木马采用了多种绕过手段，包括行为伪装（模拟正常应用程序的运行方式）^[11,12]、代码混淆（改动代码结构或内容来增加分析的难度）^[13,14]以及虚拟机逃逸（绕过沙箱或虚拟化环境的检测）^[15,16]等。这使得远控木马更难以被传统的防病毒软件和入侵检测系统识别和拦截。但与此同时，这也推动着远控木马的检测方法不断发展和更新。现有的检测方法主要可以分为基于静态文件的检测方法、基于行为分析的检测方法和基于网络流量的检测方法。

(1) 基于静态文件的检测方法



基于静态文件的检测方法通过分析应用程序的二进制文件、反汇编文件、动态链接库等静态文件的内容或结构，从中提取特征来实现木马检测。**Kakisim** 等人^[17]利用恶意软件的 PE 文件，通过将其表示为一个操作码子图结构，使用随机游走方法和词嵌入技术来自动学习恶意软件可执行文件的判别性操作码序列模式，以实现检测。**孙世淼**等人^[18]考虑相邻字节关联性和不同长度汇编指令字节分布的问题，将恶意代码的二进制文件转换为基于频率域表示的马尔可夫图像后利用深度学习进行检测。基于静态文件的检测方法主要是利用程序的二进制代码、反汇编代码等与程序功能相关的文件中提取的信息，通过各种特征工程提取特征后利用机器学习或深度学习模型实现检测，其有效性依赖于程序源文件内容，特别是在远控木马越来越多地采用加壳技术和代码混淆时^[5]，其常常导致文件内容的显著变化，基于静态文件的检测方法的准确性则会受到影响。

(2) 基于行为分析的检测方法

基于行为分析的检测方法通常通过将程序运行在虚拟环境中，监控程序运行时的行为和系统的变化来识别异常活动从而实现检测。**Canali** 等人^[19]提出了一种基于行为特征的恶意软件检测方法,从原子操作的选取和利用何种基本结构来组合两方面考虑构建检测模型。**Farinholt** 等人^[20]搭建了一个模拟真实用户环境的蜜罐系统，运行并检测 **DarkComet** 样本的行为，并通过行为重构操作者的操作序列，以获得操作者的动机、行为模式以及使用的工具。**BehradFar** 等人^[21]则结合静态文件特征与动态行为特征，通过分析 DLL 文件中相关的库函数并通过在虚拟化环境中运行木马发现其关键的行为特征，再通过两层特征选择与机器学习方法，实现高精度检测。基于行为分析的检测方法通过监控程序在执行过程中的行为以表征其实际的运行逻辑或目的主机环境资源变化。其通常需要在虚拟化环境中执行目标程序，对资源的消耗较大且耗时长，同时许多远控木马集成了虚拟机逃逸技术^[22]，它们通过检查执行环境是否为虚拟环境来实现逃避检测。

(3) 基于网络流量的检测方法

基于网络流量的检测方法通过分析远控木马服务端（即控制端）和客户端（即被控端）之间网络通信的流量，来发现远控木马不同于良性程序的异常通信模式，具体可分为基于统计特征和基于字节序列的检测方法。

基于统计特征的方法通过应用程序客户端与服务端之间传输流量的数据包大小、数据包持续时间等信息表征其通信模式，以此实现检测。**Yin** 等人^[23]选择前 20 个数据包作为分析对象，并将出入站数据字节、数据包数量等作为远控木马流量不同于正常网络流



量的有效特征。宋紫华等人^[24]以木马通信会话早期的前 5 个数据包为分析对象提取了 14 维统计特征，并训练了检测模型。Guo 等人^[25]提出了一种结合双侧特征的分阶段远控木马检测方法，将主机侧与网络侧特征相结合，并针对木马的保持存活状态和控制命令状态分别训练了一个检测模型以实现检测。王晨等人^[26]使用从远控木马上线包及其后续的少量数据包的负载大小序列和时间间隔序列中提取了三个特征，结合机器学习算法实现对远控木马的早期检测。Jiang 等人^[27]将从建立 TCP 连接的 SYN 数据包开始后相邻数据包的间隔时间小于阈值 t 秒之前的时间段定义为早期阶段，并从中提取特征，通过机器学习方法训练模型，检测远控木马会话。Jiang 等人^[28]从通信 5 分钟的数据包序列的大小、到达间隔以及标志来生成了 8 个特征，包括 6 个流特征和两个会话特征，以检测远控木马。一般来说，基于统计特征的检测方法依赖于具有区分度的特征，良好的检测准确率通常需要较长时间的流量信息，其难以兼顾检测及时性与高检测精度，且需要耗时的特征工程。

近年来，深度学习技术在多个领域取得了出色的效果，其可从原始数据中自动学习潜在特征。随着深度学习技术的广泛应用，国内外研究者也开始利用深度学习技术来构建基于字节序列的远控木马检测方法。Wang 等人^[29]直接将原始流量数据统一为长度为 784 字节，每个字节代表一个像素，生成灰度图像，输入到 CNN 模型进行训练和流量分类。Chen 等人^[30]提出了一种基于时间卷积网络的入侵检测模型，用于学习序列中每个数据包字节的特征，以充分利用其结构化信息；吴丰源等人^[31]通过将应用程序流量的包长序列、包负载长度序列和包时间间隔序列表征为融合序列，利用 Transformer 捕获流量中的时序关系，利用通信双方的前 20 个数据包实现木马检测。谷勇浩等人^[32]通过将虚拟对抗训练与 mean teacher 模型相结合，引入半监督模型用于解决无标记样本未充分利用的问题，并通过虚拟对抗用以解决模型泛化性不足的问题，实现了准确率和泛化性的提升。但现有基于字节序列的检测方法往往会为适应深度学习模型固定尺寸的输入的原因采用填充或截断的方式。由于远控木马运行初期所产生的流量较少，其数据序列较短，因此即便是微小的序列变化也可能对整体行为特征产生显著影响。这种情况下，采取填充方法将引入无关的干扰信息，而截断则将导致信息丢失，不利于获取高的检测精度。于是 Pi 等人^[33]在 2024 年利用远控木马在早期阶段产生的流量中提取的字节序列，提出了一种基于马尔可夫矩阵与深度学习的远控木马流量检测方法 RATMD，无需对字节序列进行截断。总的来说，基于字节序列的方法由于使用远控木马在其运行初期的少



量流量，使得其对于特征表示的依赖性高。此外大多数基于字节序列的方法使用深度学习构建检测模型，这使得其计算资源消耗大且依赖于训练数据的规模和多样性。

综上所述，基于静态文件的检测方法难以应对远控木马的强加密和代码混淆，基于行为分析的检测方法存在着检测耗时长，资源消耗大的问题。而现有的基于网络流量的远控木马流量的检测方法中，基于统计特征的检测方法较为依赖于特征工程，且通常需要采集较多的远控木马流量才能获取高的检测精度，因此检测及时性不佳；部分基于字节序列的远控木马流量检测方法因深度学习模型结构限制，存在输入处理过程中的信息损失和较高计算资源消耗等问题，影响其在实际部署场景中的可行性。

1.2.2 发展趋势

随着远控木马攻击技术手段的持续升级以及云计算中心等大规模网络环境中的快速发展，未来远控木马检测的研究可能会向以下几个方向展开：

(1) 对抗性防御技术：现代远控木马通过动态行为深度伪装（如模仿合法进程的 API 调用序列）、代码混淆技术以及虚拟环境检测逃逸机制等构建了多层防御规避体系。远控木马对抗性技术的复杂化，对传统检测模型的鲁棒性与可解释性提出了严峻挑战。这要求检测技术能够有效应对木马的逃避技术，如检测其在虚拟机中的逃逸行为、分析动态行为的伪装模式等。此外，随着对抗性攻击的技术逐步成熟，结合生成对抗网络（GAN）等技术增强检测系统的鲁棒性，已经成为一种重要的研究趋势。

(2) 多模态信息融合：由于单一模态特征在远控木马检测中存在固有局限性，多模态信息融合通过集成多种数据源（如静态特征、动态行为、网络流量等）能够实现对远控木马的更全面的分析，提高检测的准确性与鲁棒性。此外，尤其是在短时间内实现木马检测的需求下，单一模态的数据量通常不足以提供足够的信息，因此融合不同模态的特征有利于在短时间内获得更具判别力的特征，更好地兼顾远控木马检测的及时性和准确性。

(3) 轻量化部署：随着云计算、边缘计算等技术的快速发展，远控木马已成为这些新兴场景中的重要安全威胁。边缘节点和终端设备由于其计算能力受限且安全防护措施相对薄弱，极易成为攻击的目标。因此，远控木马检测系统的轻量化成为一个不可忽视的问题。传统的深度学习模型在此类环境中由于因计算和存储资源受限，难以实现高效运行。

因此，未来的研究应聚焦于构建轻量级的检测模型，在保证检测准确率的前提下，



尽可能降低计算资源的消耗，以满足边缘计算与终端环境中对安全检测的实时性和高效性的需求。

1.3 主要研究内容

本文在分析现有的远控木马检测方法的优缺点之后，针对现有远控木马检测方法在兼顾检测泛化性、及时性和资源消耗方面存在的不足，旨在通过分析远控木马在其运行初始阶段产生的网络流量，从中提取具有判别能力的特征，并结合合适的特征构建与机器学习算法，构建轻量化的检测模型，实现远控木马流量的高效检测与快速处置。本文的主要研究内容如下：

(1) 通过对远控木马样本与良性程序在建立 TCP 连接后初期的通信行为进行对比分析，聚焦于远控木马在其运行初始阶段的客户端行为，发现远控木马客户端通常会在连接建立后主动向服务端发送携带主机环境信息和自身配置的上线包或维持通信的心跳包或不规律的命令响应的数据包。而良性程序在相同阶段时其客户端行为与远控木马存在显著差异。在此基础上，分析并总结了适用于远控木马早期检测的流量数据包。

(2) 在上述分析的基础上，设计了一种基于流量字节短序列的远控木马检测方法。方法从 TCP 连接建立后客户端发送的前 k 个（ k 为一个小值）有载荷的 TCP 数据包中提取 TCP 包头与载荷的字节信息，并构建短序列字节序列。使用 n-gram 结合 TF-IDF 的方式生成特征向量，采用随机森林算法构建检测模型。通过在真实环境中运行远控木马样本和良性程序并采集其网络流量得到的数据集上进行实验评估，结果表明该方法在仅使用远控木马运行初始阶段中客户端发送的前 k 个有载荷的 TCP 数据包的情况下，能够实现较高的检测准确率，同时模型具备轻量化的优点，具有较高的检测效率。

(3) 基于所设计的方法，实现了一个集检测与终止于一体的远控木马早期检测系统。该系统可在用户主机中实时采集应用程序网络流量，自动进行后台分析与实时检测；当检测到异常流量时，系统将根据检测结果定位相关木马进程并自动进行终止处理，提升用户主机的网络安全防护能力。同时还生成告警邮件以及相关日志文件，支持后续的安全审查与分析。

1.4 论文组织安排

本文一共有七个章节，各章节的组织结构如下：

第一章：首先介绍了远控木马的危害，说明了远控木马检测的背景和意义。其次分



别从基于静态文件的方法、基于动态行为的方法和基于网络流量的方法对国内外远控木马检测技术研究现状进行了分析，预测发展趋势。最后对本文的研究内容和文章组织结构进行了介绍。

第二章：介绍了远控木马的基本概念以及网络流量分析技术，并对特征提取、机器学习的相关技术做介绍，为后续模型设计与系统实现奠定理论基础。

第三章：设计了一种基于流量字节短序列的远控木马早期检测模型。通过对流量序列进行分析与预处理，提取关键特征，并引入随机森林算法进行训练与预测。同时，通过实验对所构建的模型的有效性和性能进行了评估和分析。

第四章：在第三章设计的检测方法的基础上，进一步设计了一个集检测与终止于一体的远控木马早期检测系统。同时明确了系统的功能需求和非功能需求，完成了总体架构设计与模块功能划分，明确了系统的工作流程与实现目标。

第五章：详细介绍了系统的具体实现过程，包括网络流量实时捕获模块、远控木马检测与终止模块、告警提示模块、Pcap 包离线检测模块和检测报告生成模块等内容，展示了系统的技术实现细节与关键模块设计。

第六章：从功能测试和性能测试两个方面对所设计的检测系统进行了全面测试，验证了系统在实际运行中的准确性、实时性与稳定性，系统可及时且准确地检测远控木马，满足预期设计目标。

第七章：对文章中所作工作进行总结，对所设计的检测方法的不足之处做出分析，并对后续工作进行了展望，指出可进一步优化和拓展的方向。



第二章 相关技术

2.1 远控木马概述

2.1.1 远控木马的定义

远控木马是一种通过网络连接为攻击者提供对目标主机的远程完全控制权限的恶意软件。其采用 Client/Server 架构，分为服务端和客户端，攻击者可以通过服务端组件下达指令，控制部署在受害主机上的客户端，从而实现诸如文件操作、键盘记录、屏幕监控、摄像头/麦克风激活、敏感信息窃取甚至系统破坏等操作。远控木马是高级持续性威胁（APT）攻击中常见的重要工具，它具备极强的隐蔽性和持续控制能力，适合在目标系统中长期潜伏而不被察觉。与一般病毒或蠕虫不同，远控木马更注重操控的“隐性化”与“持久性”，并可实现精细化的远程管理功能，为攻击者提供几乎等同于本地物理接触的操作能力。

2.1.2 远控木马特性

远控木马在设计与实现过程中表现出以下主要技术特性：

(1) C/S 架构模式

远控木马采用客户端-服务端架构，攻击者通过远程控制端即服务端向目标主机中运行的被控端即客户端发送命令，实现对系统的远程操控。该结构具有可扩展性强、控制灵活等特点，便于攻击者批量管理多个受控终端。

(2) 反向连接机制

为绕过目标主机可能存在的网络访问限制，如防火墙或 NAT 设备等，远控木马客户端多采用反弹式连接，即由客户端主动向服务端发起连接请求，从而实现受控端对控制端的反向连接，有效提升通信成功率和隐蔽性。

(3) 基于 TCP 协议的通信方式

在通信协议选择方面，大多数远控木马采用面向连接的 TCP 协议，借助其可靠性保证命令和数据的稳定传输。

(4) 强隐蔽性与对抗检测能力

远控木马通常采用多种混淆、加密与反分析技术（如代码加壳、进程注入、Rootkit 技术等）隐藏自身存在，逃避杀毒软件、行为监测系统及静态分析工具的识别。同时，部分样本具备环境感知能力，能够检测虚拟机或沙箱运行环境，从而选择性地执行或中

止自身行为。

(5) 持久化驻留机制

为确保在系统重启后依然能够重新建立远程连接，远控木马广泛采用持久化技术，如修改注册表启动项、创建计划任务、注入系统服务进程等方式，保障其在目标系统中的长期驻留和持续控制能力。

2.1.3 远控木马通信行为

为更细致地理解远控木马的通信行为，如图 2.1，本文将现有的远控木马攻击阶段^[25]细化为以下四个阶段：建立连接阶段、信息回传阶段、命令控制阶段和保持存活阶段。在建立连接阶段，远控木马客户端与服务端建立初始通信连接；在信息回传阶段，远控木马客户端将把目标主机的环境信息及自身配置封装在数据包中通过网络发送给服务端，该类数据包通常被称作上线包；在命令控制阶段，攻击者在获取到上线包之后，根据其环境信息和木马相关配置向客户端发送指令，实施具体攻击行为；在保持存活阶段，远控木马将进入潜伏状态，周期性地向服务端发送心跳包以维持通信。

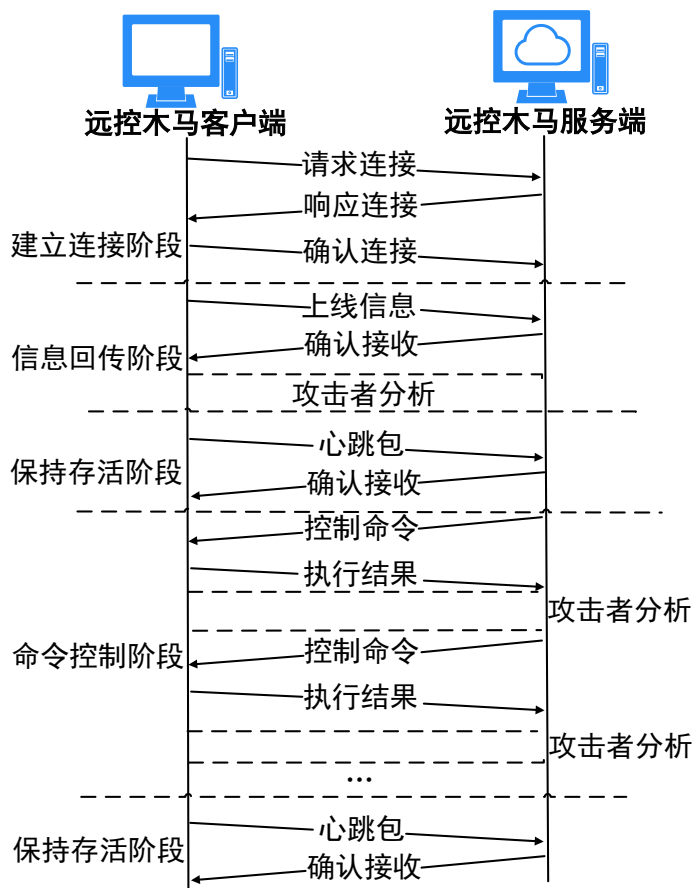


图 2.1 远控木马通信阶段



2.2 网络流量分析技术

2.2.1 流量采集方式与工具

网络流量分析的第一步是准确获取网络中的数据包，采集到的数据是否全面、真实、有效，直接影响后续的特征提取和分析效果。根据采集策略的不同，流量采集方式通常分为被动采集和主动采集两类。被动采集主要依赖已有网络基础设施，在不干扰正常通信的前提下对网络中的数据进行旁路监听。这种方式通常借助网络设备的端口镜像功能，将指定端口或 VLAN 中的全部数据复制到监控端口，供采集设备接收分析。该方式部署相对简洁，对生产网络影响较小，因此被广泛用于交换机、路由器等边界设备的数据采集。另一种常见的被动采集手段是使用网络流量镜像器或硬件中间件，将过往的物理数据流复制到监听系统中，实现透明的数据捕获。主动采集通常部署于关键业务节点或边缘终端上，采用专门的网络探针进行数据获取。探针设备通常具备高速包捕获能力，并内置一定的数据解析与处理逻辑，可直接分析网络流、协议交互和异常行为。该方法适用于对指定业务流、特定用户行为或安全威胁进行精准分析的场景，但同时对硬件资源、网络结构和配置要求更高。

在具体的工具实现层面，不同的流量采集工具适配不同的网络规模与分析需求，常用的流量采集工具包括 Wireshark、tcpdump 和 NetFlow 等。Wireshark 是一款功能强大的网络协议分析工具，支持对网络数据包的实时捕获、协议字段的深入解析、流量可视化展示以及过滤查询等操作。它在教学培训、协议调试、故障定位和网络取证等方面被广泛应用，尤其适合需要交互式查看和深入理解网络通信内容的场景。其命令行工具 tshark 也可用于嵌入式采集任务，具备一定自动化能力。然而在处理高频流量或要求低延迟的实时系统中，Wireshark 更适合作为辅助分析工具而非主力采集模块。tcpdump 则是一款运行于命令行环境下的轻量级抓包工具，具备强大的脚本控制能力和灵活的过滤语法，适合部署于服务器、终端等资源受限的环境中进行长期监控与自动化任务调度。NetFlow 是一种以流为单位的网络流量监控协议，能够在不抓取完整数据包的前提下，对网络中各类连接行为进行统计和汇总，常用于大规模网络环境下的流量趋势监控与行为分析。

尽管上述工具在各种网络环境中表现出色，但在资源受限的终端设备或在需要与分析模块深度集成的场景下，其部署与使用存在一定的门槛。在此类场景下，Scapy 提供了一种更为灵活的解决方案。Scapy 是基于 Python 的网络封包操作库，支持多种网络协



议的数据包构造、发送、捕获与解析，具备良好的可编程性与扩展性。其核心的流量采集能力主要通过调用 `sniff()` 函数实现，该函数允许用户对本地网络接口进行实时监听，并结合过滤表达式与回调函数实现对数据的即时处理。**Scapy** 在底层通过操作系统提供的原始套接字或基于 `libpcap` 的接口进行数据包的直接访问，其中在类 Unix 系统中通常结合 Berkeley Packet Filter (BPF) 机制完成高效过滤，在 Windows 平台则依赖 WinPcap 或 Npcap 驱动完成底层数据采集。**Scapy** 具备部署灵活、资源开销小和使用门槛低等优势，适合开发人员进行定制化的网络采集任务，便于快速调整采集逻辑，直接嵌入特征提取与预处理模块中，形成闭环的数据采集与分析流程。

2.2.2 网络流量的表示方式

原始采集到的数据包往往呈现为结构复杂、数量庞大的非结构化信息，难以直接用于行为分析或特征建模。因此，在进入后续处理前，必须对网络流量进行合理的表示。网络流量的表示方式通常可以依据粒度的不同，划分为数据包、流、会话多个层次。数据包是网络中传输的最小单元，包含从链路层到应用层的多层协议信息，能够完整保留通信的细节，适合用于协议分析和细粒度内容检测，但由于单个数据包缺乏上下文关联，在分析整体通信行为时作用有限。相比之下，流是由具有相同五元组（源 IP、目的 IP、源端口、目的端口、协议）的连续数据包构成，表示一次单向的通信过程。流的表示方式能更清晰地反映通信的整体结构，是目前网络流量分析中最常用的建模单位。而会话则通常采用三元组（通信双方的 IP 地址和协议类型）来表示，代表一次完整的应用层交互，包含双向的多个数据流。会话层的表示更适合用于分析用户行为、识别应用类型等。总体来看，不同层次的表示方式在细节保留和处理效率之间有所不同，合理选择适合的表示层次，有助于提升网络流量分析的准确性和性能。

2.3 特征提取技术

在原始网络流量中存在大量冗余信息，直接用于模型训练容易导致模型性能不佳。因此，需要将原始数据转换为能表征流量行为的特征向量。本系统使用 **n-gram** 模型和 TF-IDF 特征加权方法进行特征提取，以下是对这两种技术的介绍。

2.3.1 n-gram 模型介绍

n-gram 是一种通过局部上下文建模序列的方法，能够从连续序列中挖掘短程依赖模式。在网络流量分析中，**n-gram** 模型可以用于处理数据包负载内容或协议行为序列。其基本原理是假设序列中的当前状态仅依赖于前 $n-1$ 个状态，通过构造长度为 n 的滑动窗



口，可从原始序列中提取所有可能的 n -gram 片段，最终构建特征向量。例如，将字节序列[a,b,c,d]以 $n=2$ 分割后可得[ab,bc,cd]，这些片段组成的集合可构建词袋模型，并转化为定长向量，用于后续分类器的输入，提取出频繁出现的模式。这种方法能够捕捉到短期的上下文依赖特征，对于恶意行为识别、协议分析等任务具有良好的适应性。

2.3.2 TF-IDF 模型介绍

为了进一步提升模型对特征重要性的区分能力，本文进一步在 n -gram 生成的基础上引入了 TF-IDF 加权机制。TF-IDF（词频-逆文档频率）是一种常用的文本特征加权方法，用于衡量文本中词汇的重要性。其也可以迁移应用于网络流量分析，将数据流看作“文档”，将其中的 n -gram 片段视为“词”。TF 衡量某个 n -gram 在该流量中的频繁程度，而 IDF 则衡量该 n -gram 在所有流量中出现的普遍性，TF-IDF 通过计算某个 n -gram 在流中的频率（TF）以及该 n -gram 在所有流中出现的稀有程度（IDF），来衡量该特征的重要性，经 TF-IDF 加权后的特征向量具有更高的区分能力，使得模型在检测远控木马时能更有效地识别出行为上的细微差异。

2.4 机器学习

2.4.1 机器学习的概念和模型

机器学习是一种通过训练数据自动构建预测或分类模型的技术。在网络流量分析领域，机器学习可以用来识别异常行为、区分正常与恶意流量等。根据学习方式的不同，机器学习可分为监督学习、无监督学习和半监督学习等类型。其中，监督学习通过使用已标注数据进行训练，能够有效构建针对特定任务的分类器，是目前最主流的应用方式。本文采用监督学习模型对网络流量进行建模，训练集中样本已被人工标注为“良性”或“木马”，从而支持分类器的有效训练。常见的模型包括支持向量机（SVM）、决策树、神经网络、K 近邻（KNN）等，本文选择随机森林方法来建立检测模型，其介绍如下。

2.4.2 随机森林介绍

随机森林是一种集成学习方法，由多个决策树组成，其基本原理是通过多个决策树的组合，提高整体模型的稳定性与精度。其基本思想是在训练阶段引入两种随机性：一是通过自助采样法对训练集进行有放回采样，生成多个子样本集，每棵树分别在这些不同子集上进行训练；二是在每次节点划分时，从所有特征中随机选取一个子集，仅在该子集上选择最优划分特征。这种设计不仅提高了模型的多样性，降低了模型之间的相关性，还有效防止了单一模型因过拟合而导致的性能下降问题。在预测阶段，随机森林通



过对所有子模型的投票结果进行加权汇总，从而得出最终分类决策，具有极高的稳定性与鲁棒性。在网络流量分析中，随机森林以其鲁棒性强、对噪声不敏感、训练速度快等优点被广泛应用于入侵检测、流量分类等任务。其输出结果是所有决策树投票的综合结果，具有较强的稳定性。

2.5 本章小结

本章介绍了远控木马的概念、典型特性以及通信行为模式，同时针对网络流量分析这一基础环节详细阐述了常见的流量采集工具与网络流的表示方式，介绍了特征提取以及机器学习分类的关键技术，为下一步建立检测模型奠定基础。



第三章 基于流量字节短序列的远控木马早期检测方法

3.1 远控木马流量序列分析

由 2.1.3 节中的远控木马通信流程可知，远控木马在建立连接阶段以及信息回传阶段尚未对目标主机实施直接攻击，其行为多以获取信息为主，木马在进入命令控制阶段后才会对目标主机执行攻击命令。因此，若能在远控木马信息回传阶段结束前实现对远控木马的检测，将对目标主机的防护具有很大意义。

为实现对远控木马的早期检测，需要采集远控木马初始阶段的流量来实施检测。在远控木马通信的前两个阶段（即建立连接阶段与信息回传阶段），其通信行为的主导为远控木马客户端。客户端在建立连接后，将主动向服务端发送包含目标主机环境信息和自身配置的上线包，而服务端此时未掌握目标主机的相关信息，其行为较为被动，主要为接收连接请求或等待信息回传，难以提供有效的利于检测的信息。相较之下，客户端在此期间行为更能够反映远控木马的攻击意图。因此，本文聚焦于远控木马运行初始阶段的客户端通信行为。此外，由于携带有效信息的数据包为具有载荷的数据包，且远控木马为保证通信可靠性，常以 TCP 协议进行数据传输，故本文以客户端发往服务端的有载荷的 TCP 数据包作为分析对象。为进一步明确分析范围，本文将客户端在建立连接后客户端发送的前 k 个具有载荷的 TCP 数据包作为分析对象，其中 k 为一个较小的固定值。而由于木马设计者对信息采集和隐蔽性策略的不同需求，因此相同 k 值下，不同的远控木马所处的阶段可能不同，具体可分为以下三种情况：

(1) 信息回传阶段，此时 k 值小于等于上线包数量，用于分析的数据包为上线包，包含主机操作系统版本、用户名、是否安装杀毒软件等系统环境信息以及木马自身配置信息等，上线包的内容直接反映木马对目标主机环境的收集与潜在攻击意图，表现出较强的区分特征，具有较高的检测价值。

(2) 保持存活阶段，此时 k 值大于上线包数量，用于分析的数据包除上线包外还包含少量由客户端发送给服务端的用于保持连接的心跳包，此类数据包通常只携带简单的标识符、时间戳或状态码，用于告知服务端客户端仍然在线，在大多数情况下不涉及复杂的交互，因此负载较小，即使存在少量心跳包，其在整体特征中的占比有限，对模型检测效果的影响较小。

(3) 命令控制阶段，此时 k 值大于上线包数量，用于分析的数据包除上线包外还包



含少量客户端向服务端发送的命令执行后的回复数据包。此类数据包的内容取决于攻击者发送的命令，具有不规律性。

相比之下，良性程序在建立连接后客户端同样会向服务端发送数据包，但其通信行为呈现出显著不同的特点。良性程序普遍使用标准化协议（如 HTTPS、TLS 等），客户端发送的数据包多用于资源请求或身份认证，数据格式规范、结构稳定，且不会主动回传本地系统信息。这些数据通常嵌套在标准协议字段中（如 HTTP 的 Host、User-Agent 等），在通信早期的数据形态上与远控木马存在本质差异。

综上所述，即使不同远控木马在相同 k 值下所处的通信阶段不同，其客户端通信行为相比良性程序客户端在相同阶段呈现出的特点仍具有显著差异。因此，基于客户端建立连接后发送的前 k 个有载荷的 TCP 数据包进行分析建模，具有良好的检测优势。特别地，由于所使用的 k 值为一个小值，尽管少量远控木马已进入命令控制阶段，其仍处于较早的阶段，具有良好的及时性。

3.2 基于流量字节短序列的远控木马早期检测方法

本文设计了一种基于流量字节短序列的远程控制木马早期检测方法（RAT early detection Method based on short traffic Sequence, RATMS），RATMS 的整体框架如图 3.1 所示。该方法聚焦于远控木马运行初始阶段的客户端通信行为，基于远控木马建立连接后其客户端向服务端发送的前 k 个有载荷的 TCP 数据包，通过从中提取的流量字节短序列判断其是否为远控木马流量，方法包括以下三个步骤：

步骤 1：流量预处理。RATMS 在流量预处理阶段采集远控木马和良性程序在运行初始阶段所产生的网络流量，并从中提取字节序列，以获得流量字节短序列。具体来说，字节序列源于应用程序客户端首次建立连接后，向其服务端发送的前 k 个具有载荷的 TCP 数据包的 TCP 包头与 TCP 载荷。

步骤 2：特征计算。RATMS 使用 n-gram 算法分割流量短序列并通过 TF-IDF 算法计算出流量字节短序列相应的特征向量。

步骤 3：模型训练和检测。RATMS 使用在特征计算阶段所得到的特征向量结合机器学习算法构建一个远控木马流量检测模型，用于区分远控木马流量与良性程序流量。

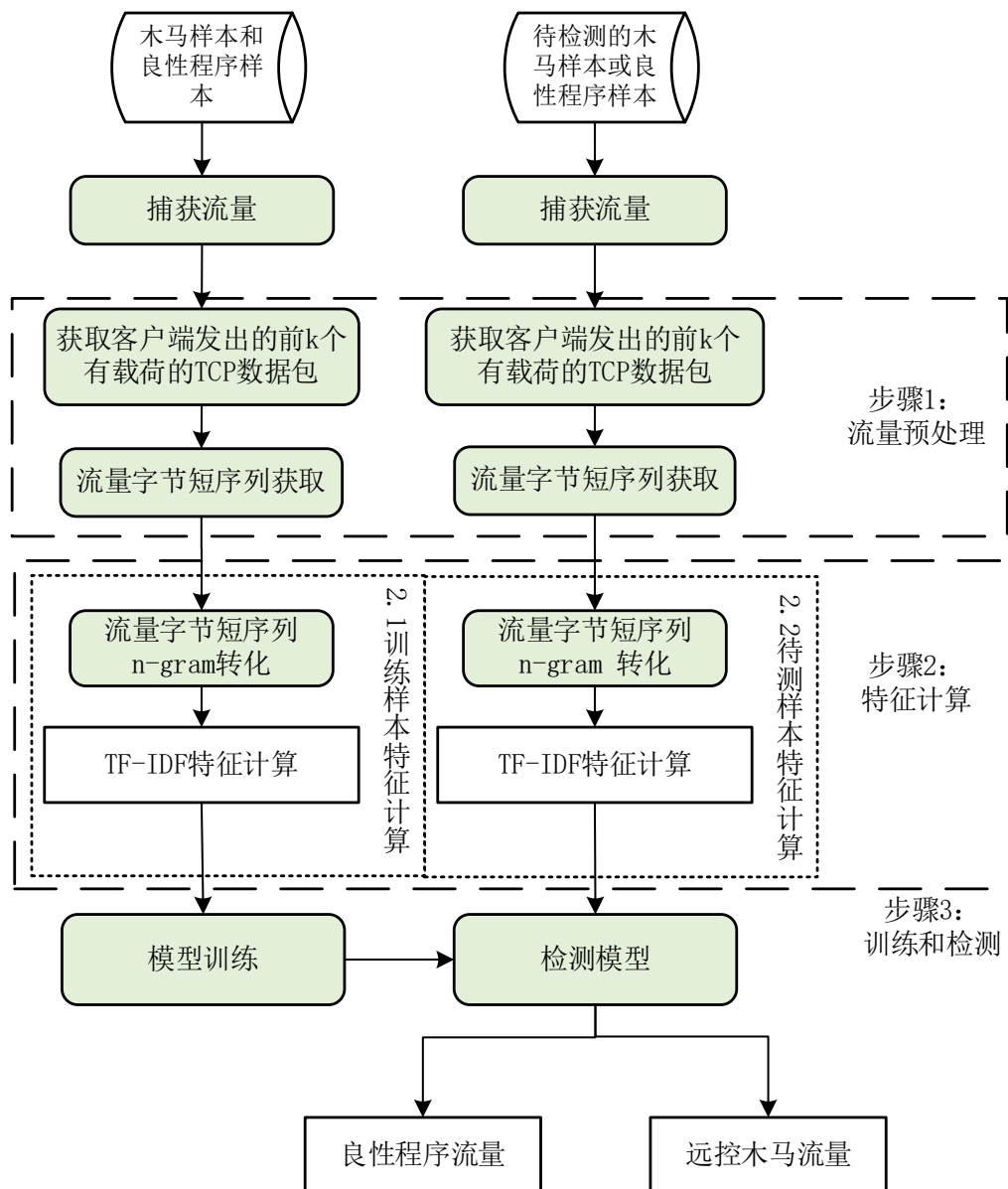


图 3.1 基于流量字节短序列的远控木马早期检测方法框架

3.2.1 流量预处理

流量预处理阶段旨在定位应用程序的运行初始阶段，并从收集到的网络流量中提取字节序列。与文献[13]一样，本文将会话作为基本检测单元，会话为一组在连续时间段内具有相同的三元组{源 IP、目标 IP、传输级协议}的数据包。如第 3 章所述，为了实现远控木马流量检测，本文收集应用程序客户端在首次与服务端建立 TCP 连接后，发送给服务端的前 k 个带有 TCP 载荷的数据包，并将这些数据包作为分析对象。并从中提取其 TCP 包头与 TCP 有效载荷的字节序列，这些字节序列包含了应用程序流量的字节信息，反映了应用程序在运行初始阶段的关键特征。

3.2.2 特征计算

特征计算步骤旨在通过采用适当的流量表征方式，有效地保留在流量预处理步骤中收集的流量字节信息。为了实现这一目标，RATMS 采用 *n-gram* 算法分割流量序列并通过 TF-IDF 算法计算出相应的特征值。

RATMS 使用 *n-gram* 算法来提取流量序列中字节之间的相邻关系。具体来说，本文将提取出来的流量的字节序列 $Bytes = \{Byte_i: i > 0\}$ 视为一个马尔科夫链，并假设流量字节 $Byte_i$ 的出现概率只依赖于前面 n 个字节。RATMS 通过使用 *n-gram* 序算法生成包含连续 n 个字节的子序列，这些子序列可以捕获字节之间的局部特征，表征字节间的相邻关系，为后续的特征计算提供基础。

在使用 *n-gram* 对流量序列处理后，RATMS 采用了自然语言处理中的 TF-IDF 算法来计算特征值并生成相应的特征向量，从而进一步提取在流量序列中具有显著性和代表性的特征。RATMS 采用式 3-1 计算流量 *n-gram* 序列的 TF 值。

$$TF(g_i, s_j) = \frac{c(g_i, s_j)}{\sum_k c(g_k, s_j)} \quad (3-1)$$

其中 $TF(g_i, s_j)$ 代表在流量 *n-gram* 序列 s_j 中流量 *n-gram* 词条 g_i 出现的频率。 $c(g_i, s_j)$ 表示流量 *n-gram* 词条 g_i 在流量 *n-gram* 序列 s_j 中出现的次数。 $\sum_k c(g_k, s_j)$ 表示流量 *n-gram* 序列 s_j 中所有流量 *n-gram* 词条出现的总和，其中 k 为序列 s_j 中不同 *n-gram* 词条的种类数。在计算出一个词条的频率后，通过式 3-2 计算其 IDF 值：

$$IDF_{g_i} = \log_2 \left(\frac{D}{d} \right) \quad (3-2)$$

其中， IDF_{g_i} 描述的是某一特定流量 *n-gram* 词条 g_i 在全部流量 *n-gram* 序列中的重要程度， g_i 越少， IDF_{g_i} 越大，说明流量 *n-gram* 词条 g_i 越重要，具有良好的类别区分能力。 D 表示流量 *n-gram* 序列总数， d 表示包含流量 *n-gram* 词条 g_i 的流量 *n-gram* 序列的数量。之后采用式 3-3 计算其 TF-IDF 值：

$$TF-IDF_{g_i, s_j} = TF(g_i, s_j) \times IDF_{g_i} \quad (3-3)$$

$TF-IDF_{g_i, s_j}$ 表示流量 *n-gram* 词条 g_i 在流量 *n-gram* 序列 s_j 上的 TF-IDF 值。

3.2.3 模型训练与检测

本步骤旨在构建一个能够自动且有效地学习在特征计算阶段所得到的特征向量的远控木马流量检测模型。与文献[24]等相关工作类似，本文将远控木马流量检测视为一个二分类问题。首先将上一阶段提取到的特征向量添加类别标签（0 对应良性程序，1 对应远控木马），得到训练集 $X_{train} = \{(Q_1, Y_1), \dots, (Q_m, Y_m)\}$ ， X_{train} 是 m 个带标签的会话集，其中 $Q_1, \dots, Q_m$ 代表样本特征向量， $Y_i \in \{0, 1\}$ 为对应的类别标签。之后运用机器学习分类算法在训练集 X_{train} 上训练远控木马流量检测模型。在检测阶段，通过经过训练的模型，每个待检测的会话都将被归类为远控木马流量或良性程序流量的类别标签。

3.3 实验结果与分析

3.3.1 实验环境与数据集

RATMS 基于 Python3.10 结合 scikit-learn 实现。本文在 Windows10 上开展实验，并使用 11th Gen Intel(R) Core(TM) i7-11800H 2.30GHz CPU 和 16GB RAM 对模型进行训练。考虑到模型复杂度和训练耗时，本文根据经验选择了 RATMS 中的超参数。RATMS 使用的超参数为：随机森林算法中决策树数量为 200，树的最大深度为 8，n-gram 的 n 为 4。

为使得本文的测试最大限度的模拟真实的远控木马流量检测场景，在构建数据集时，特别注意选取让用于测试的样本完全不同于用于模型训练的样本，这使得测试集中的样本可视为训练样本的“未知样本”，从而更好地评估本方法对未知远控木马的检测能力。具体来说，本文在不同运行环境中运行远控木马样本与良性程序样本，并收集它们在运行中产生的流量，以构建本文用于训练与测试的流量数据集。具体来说，本文使用 Windows7（32 位）、Windows7（64 位）、Windows8、Windows10 Professional、Windows Server 2019 或 Windows10 Education 上运行的虚拟机来运行远控木马客户端和良性程序，并将远控木马服务端部署在运行 Windows Server 2019 的云服务器上，使用 Wireshark 从运行客户端的主机上捕获网络流量。图 3.2 展示了本文在构建数据集时使用的实验拓扑结构。

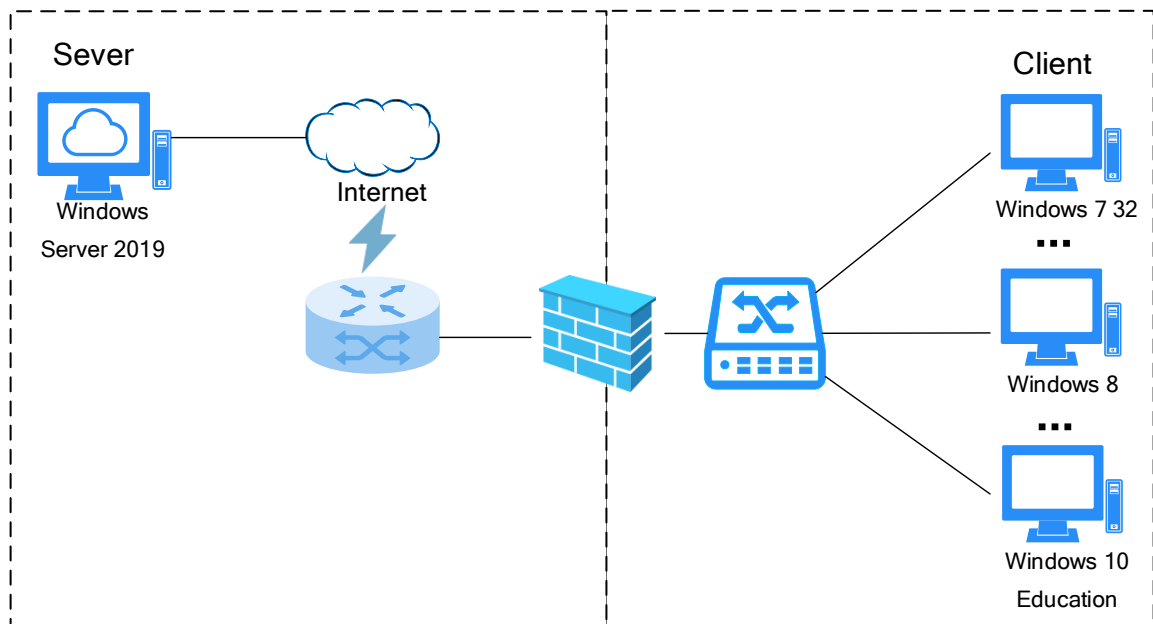


图 3.2 实验拓扑图

本文基于收集到的网络流量构建了训练集和测试集。为评估方法对未知木马的检测能力，测试集中引入了与训练集中不同的 RAT 样本。训练与验证样本在运行环境集 {Windows 7 (32 位)、Windows 7 (64 位)、Windows 8、Windows 10 Professional、Windows server 2019} 中采集，测试样本则在环境集 {Windows 10 Education} 来采集。表 3.1 中列出了实验中使用的共 45 个 RAT 样本。编号 1 至 30 的样本活跃时间为 2011 年至 2018 年，用于生成训练集的远控木马流量；编号 31 至 45 的样本活跃时间为 2018 年至 2024 年，用于生成测试集中的远控木马流量。考虑到远控木马常根据攻击者意图对受害设备实施不同操作，因此对每个 RAT 样本进行了多次运行，并在其与服务端建立首个 TCP 连接后，随机选择执行攻击指令或保持空闲，以生成多样化的 RAT 会话，每个会话时长约 5 分钟。此外，实验还包含了多类良性程序，如网页浏览、即时通信、云服务、安全软件、视频直播、在线办公、网络购物以及在线订餐等。编号为 1 至 30 的良性程序样本作为训练集，编号为 31 至 45 的良性程序样本作为测试集。同样对这些良性程序进行了多次运行并随机操作，每个会话时长约 5 分钟，以生成多样化的良性程序流量会话。最终，训练集由表 3.1 中编号 1 至 30 的 RAT 样本所成的 948 条会话，与表 3.2 中编号 1 至 30 的良性程序样本所生成的 1000 条会话组成。测试集则由表 3.1 中编号 31 至 45 的 385 条 RAT 会话与表 3.2 中编号 31 至 45 的 550 条良性会话构成。各类会话数量及其分布情况详见表 3.3。



表 3.1 远控木马样本

序号	远控木马	序号	远控木马	序号	远控木马
1	AsyncRAT v0.5.6D	16	DarkComet RAT v4.0	31	BlackWorm v6.0.0
2	Babylon RAT 1.5.1.0	17	DarkComet RAT v5.3	32	Dangrous RAT 2020
3	Babylon RAT 1.6.0.0	18	Darktrack v4.0	33	DcRat 1.0.7
4	BifrostRAT 0.5	19	DRAT 2011 V5.1	34	Eagle Monitor v3.2.4.0
5	BlackWorm v4.1	20	Greame RAT v1.9	35	GhostDZ RAT v1.1
6	BlackNix RAT	21	KilerRat v10.0.0	36	Hiddenz'sHVNC v4.0.0.2
7	Bozok 1.4.3	22	KnightRAT v1.0.0.0	37	Horus Eyes Rat v0.2.1.0
8	Bozok 1.5.1	23	L6-RAT Beta 1	38	Imperium RAT
9	BX RAT V1.0	24	Mega RAT 1.5 Beta	39	Mosaique Rat
10	Cloud Net 1.7	25	MLRAT 1.0.0.5	40	NjRAT 0.7D
11	CobianRAT v1.0.40.7	26	MQ5 RAT	41	RPS420 RAT
12	CometRat v0.1.4	27	NanoCoreRAT 1.2.2.0	42	Venom RAT v6.0.3
13	ctOs 1.3.0.0	28	NingaliNET v1.1.0.0	43	Venom Remote Control
14	CyberGate v1.07.5	29	NJRAT 0.5D	44	WARZONERAT 1.01
15	Daleth RAT 1.0	30	NovalLite v3.0	45	WH-RAT v1.0.1

表 3.2 良性程序样本

序号	良性程序	序号	良性程序	序号	良性程序
1	360 Safe	16	Kdocs	31	Alipay
2	360 Security Browser	17	Keep	32	Baidu Translate
3	360 Yunpan	18	Kingsoft Powerword	33	Bilibili
4	360zip	19	KuGou Music	34	CSDN
5	AliMail	20	Kuwo Music	35	Eleme
6	Aliyun	21	Ludashi	36	iFLYTEK Input
7	Baidu Netdisk	22	Mail Master	37	Jingdong
8	Cbox	23	MangoTV	38	Meituan
9	Chinatelecom Cloud	24	Meitu	39	Taobao
10	Chrome	25	Microsoft Edge	40	Tecent Sports



表 3.2 良性程序样本（续）

序号	良性程序	序号	良性程序	序号	良性程序
11	DingTalk	26	Migu	41	TikTok
12	Firefox	27	NetEase Cloud Music	42	Weibo
13	Gaode Map	28	Opera Web Browser	43	Xiguashipin
14	Huya Live	29	QQ Game	44	Yangshipin
15	IQIYI	30	QQ Music	45	Zhihu

表 3.3 实验数据集分布

序号	训练集	测试集	总和
良性程序流量	1000	550	1550
木马流量	948	385	1333
总和	1948	935	2833

3.3.2 评估指标

本文使用四个常见评价指标 Accuracy, Precision, Recall, F1-score 对 RATMS 的实验效果进行评估。四个指标的具体的计算方法如式 3-4、式 3-5、式 3-6 与式 3-7 所示。

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (3-4)$$

$$Precision = \frac{TP}{TP + FP} \quad (3-5)$$

$$Recall = \frac{TP}{TP + FN} \quad (3-6)$$

$$F1 - score = \frac{2 * Precision * Recall}{Precision + Recall} \quad (3-7)$$

这些指标可以通过 TP、TN、FP、FN 计算得到，其中 TP 表示远控木马流量被正确检测的数量，FN 表示远控木马流量被检测为良性程序流量的数量，FP 表示良性程序流量被检测为远控木马流量的数量，TN 表示良性程序流量被正确检测的数量。

3.3.3 实验结果与分析

由于本文使用远控木马建立连接后客户端向服务端发送的第 k 个有载荷的 TCP 数据包作为检测对象。因此 k 值决定了可用于 RATMS 分析的网络流量。所以， k 不能太小也不能太大。如果 k 太小，则可供分析的网络流量较少，不利于准确地检测远控木马；如果 k 太大，则此时远控木马已经在命令控制阶段持续了一段时间。表 3.4 给出了 RATMS



在不同 k 值下使用机器学习分类算法分别构建模型所得到的实验结果。

从表 3.4 可以看出本方法在 $k=2$ 时效果相比 $k=1$ 和 $k=3$ 时效果更好，这是因为当 $k=1$ 时所使用的序列较短，而 $k=3$ 时引入了更多非规律的流量字节序列。此外，可发现使用随机森林的方法时检测效果最好，检测准确率可达 97.3%。这是因为随机森林算法是一种包含多个决策树的分类器，其输出的类别由各个决策树输出类别的众数决定而不容易产生过拟合，使得方法检测准确率高，泛化能力强，对于未知木马仍然具有较高的检测率。

表 3.4 5 种算法在不同 k 值下 RATMS 对未知木马测试集的检测结果

算法 \ k	1				2				3			
	Accuracy	Precision	Recall	F1	Accuracy	Precision	Recall	F1	Accuracy	Precision	Recall	F1
SVM	0.884	0.986	0.730	0.839	0.864	0.996	0.673	0.803	0.860	1.000	0.660	0.795
DT	0.952	0.972	0.909	0.940	0.965	0.978	0.935	0.956	0.964	0.976	0.935	0.955
NB	0.789	1.000	0.488	0.656	0.826	1.000	0.577	0.732	0.840	1.000	0.610	0.758
KNN	0.744	1.000	0.379	0.550	0.752	1.000	0.397	0.569	0.774	1.000	0.452	0.623
RF	0.964	0.978	0.933	0.955	0.973	0.995	0.940	0.967	0.966	1.000	0.917	0.957

为了进一步评估 RATMS 的检测精度，本文将 RATMS 与第二章中介绍的两类基于网络流量的检测方法进行了比较，即基于特征的检测方法和基于字节的检测方法。方法[27]和方法[28]都是基于特征的方法。方法[27]分析了其定义的早期阶段中通信行为的特征并采用随机森林算法获得了最佳的检测结果。方法[28]从 5 分钟的通信流量中提取特征，并使用极端梯度提升算法进行分类。为了比较不同方法在相近流量用量情况下的检测性能，同时不影响方法有效性的条件下，实验中对于非早期检测方法[28]通过使用由应用客户端建立的首次 TCP 连接与出现第五个 TPS（TPS 为服务端发往客户端的有载荷的 TCP 数据包）期间的通信流量进行测试。对于早期检测方法基于字节的方法和方法[33]使用客户端首次建立连接与出现第 2 个 TPS 期间的流量进行测试。基于字节的方法从应用程序客户端首次建立 TCP 连接到其服务端发送第 2 个 TPS 之间通信的流量中提取 TCP 服务数据包的 TCP 有效载荷的字节序列，并生成一个用字节填充的 256×256 的矩阵，其中每个字节的值都被除以 255，以使得矩阵元素的值在范围[0,1]内。方法[33]将从应用程序的客户端首次建立 TCP 连接到其服务端发送第二个 TCP 服务包期间的 TCP



服务包中提取的 TCP 有效载荷的字节序列，表征为大小为 256×256 的字节转移概率矩阵，并利用卷积神经网络架构来构建远控木马流量检测模型。方法[29]通过将原始流量数据统一为长度为 784 字节，然后生成灰度图像直接输入到 CNN 模型进行训练和流量分类。表 3.5 给出了 RATMS、方法[27]、方法[28]、方法[29]、基于字节的方法和基于方法[33]在相同测试集上的检测结果。

表 3.5 不同方法的测试结果

检测方法	Accuracy	Precision	Recall	F1-score
方法[27]	0.915	0.958	0.831	0.890
方法[28]	0.964	0.956	0.956	0.956
方法[29]	0.835	0.867	0.709	0.780
基于字节的方法	0.906	0.993	0.777	0.872
方法[33]	0.920	0.987	0.816	0.893
RATMS($k=2$)	0.973	0.995	0.940	0.967

从表 3.5 所示，可以看出，本方法在 Accuracy、Precision 与 F1-score 上均取得了对比方法中是最优。虽然方法[28]在 Recall 指标上略高于本文方法，但其检测所使用的流量多于本文方法，而本文方法仅基于客户端与服务端建立连接后发往服务端的前两个有载荷的 TCP 数据包可获得接近的召回率，展现出更强的早期检测能力。此外，基于字节的方法与方法[33]虽在 Precision 上表现较好，但在 Recall 上均明显低于 RATMS，说明其存在一定的漏报风险。因此，从实验结果可知道本文方法能够在远控木马的运行初始阶段，通过少量数据包实现远控木马流量的检测。

此外，表 3.6 给出了不同方法在测试集上的检测效率，分别给出了不同方法在测试集单个样本上的特征生成/图像生成、检测时的平均耗时以及特征生成/图像生成时的最大内存占用，从表中可看出本方法 RATMS ($k=2$) 在特征生成/图像生成和检测的耗时以及特征生成/图像生成的内存占用上均展现出较好的效率优势。具体而言，RATMS 的特征生成/图像生成的平均耗时仅为 1.01ms，远低于其他方法，体现了其在特征生成/图像生成阶段的效率，同时其检测耗时亦处于较低水平。此外，方法在内存开销上也处于较低水平，说明本方法在保障检测效果的同时仍然具有较高的检测效率。



表 3.6 不同方法在测试集上的检测效率

性能指标 检测方法	平均时间/ms		最大内存占用/MB
	特征生成/图像生成	检测	特征生成/图像生成
方法[27]	31.31	3.91	0.04
方法[28]	35.65	1.89	0.41
方法[29]	1569.27	2.11	494.86
基于字节的方法	35.95	90.01	44.51
方法[33]	64.03	59.11	62.36
RATMS($k=2$)	1.01	3.64	0.16

3.4 本章小结

本章在分析远控木马运行初始阶段的通信行为，设计了一种基于流量字节短序列的远控木马检测方法。将远控木马客户端首次建立连接后向服务端发送前 k 个有载荷的 TCP 数据包作为分析对象，从中提取流量字节序列作为建模依据，并使用随机森林算法实现检测。实验结果表明，本章方法 RATMS 能够在短时间内实现对未知远控木马的快速检测，同时相比现有的方法具有较好的及时性和检测效率，具有较强的实用价值。



第四章 基于流量字节短序列的远控木马早期检测系统分析与设计

4.1 系统需求分析

4.1.1 功能需求

本系统旨在通过分析应用程序服务端与客户端之间传输的网络流量所承载的字节信息来实现远控木马流量的检测，进而实现远控木马在用户主机中的定位与终止，从而确保网络安全性并降低潜在攻击风险。以下为本系统主要的功能需求。

(1) 实时网络流量捕获与处理功能

系统需具备实时监听网络流量的能力，并能够利用底层网络接口进行用户主机中的流量捕获。同时，系统应对捕获的数据包进行筛选，过滤掉无关及冗余数据，仅保留可能涉及远控木马通信的流量。此外，系统应支持深度数据包解析，提取数据包中的源 IP 地址、目的 IP 地址、端口号、协议类型及数据负载。该功能为系统提供必要的数据库基础，以确保检测模块能够高效、准确地识别潜在攻击。

(2) 远控木马流量检测与自动终止功能

系统需要通过分析在网络流量捕获阶段所捕获到的应用程序服务端与客户端之间通信的少量数据包，在远控木马运行初期就准确检测出远控木马流量。为保护用户主机安全，系统还应在检测到远控木马流量之后，对远控木马执行处置操作，结合流量源 IP 信息定位到远控木马在用户主机中的进程，并通过调用系统 API 强制终止该进程及其子进程，有效阻断木马攻击，防止进一步的数据泄露或系统控制风险。

(3) 邮件告警功能

当检测到远控木马流量时，系统应自动生成告警邮件并发送至指定邮箱。邮件内容需涵盖检测时间、木马特征、受影响主机信息、处置结果及完整的检测报告附件。该功能可以告知用户系统对于检测到的木马流量做出的相关处置，不仅可在用户不在主机旁时提供远程告警，还可避免攻击者篡改或删除本地日志，确保安全事件的完整记录，为后续审计提供依据。

(4) Pcap 包离线检测

系统 Pcap 包离线检测模块用于支持系统对用户上传的 Pcap 文件进行静态分析。该模块支持历史流量回溯检测，适用于非实时场景下的远控木马流量识别与分析。

(5) 检测报告生成功能



系统应在每次检测到远控木马流量后，自动生成结构化检测报告。报告需包含远控木马服务端 IP 地址、远控木马建立连接时间、持续时间、进程信息及系统处置结果。该功能可以帮助用户或者安全分析人员更好地了解所发生的远控木马攻击的情况，识别潜在的安全风险，并采取必要的对策应对后续威胁。

4.1.2 非功能需求

(1) 响应时间：系统需在检测到远控木马流量后 2 秒内完成关键处置任务，包括进程终止、告警生成及邮件发送等，以实现快速响应，最大程度降低远控木马的危害，保障系统安全性。

(2) 系统稳定性：系统需具备长时间稳定运行的能力，能够适应复杂攻击场景，保持高效运作。为此，系统应优化资源管理，以防止因高负载导致的崩溃或性能下降。

(3) 系统可扩展性：系统应采用模块化架构，以适应新的功能需求，支持后续功能扩展。

(4) 可维护性：系统易于维护和修改，包括代码的可读性、可扩展性、可测试性等方面的要求，以便于后续的系统维护和改进。

4.2 系统总体设计

4.2.1 设计原则

本系统在设计过程中遵循以下关键原则，以确保其在功能性、安全性、稳定性和可维护性方面的可靠性与实用性：

- (1) 实时性原则：系统应具备高效的流量处理能力，能够实时捕获和分析网络流量，在远控木马运行初期即完成识别和响应，以达到早期检测与快速处置的目的。
- (2) 准确性原则：系统需具备高识别准确率，能够有效区分正常流量与远控木马流量，避免误报与漏报，从而保障系统的可信性和实用性。
- (3) 安全性原则：系统需最大限度保障用户主机的安全，在发现异常通信时立即执行处置动作，防止远控木马造成更深层次的渗透、控制或数据窃取，同时保证日志与告警信息的完整性不被篡改。
- (4) 可扩展性与可维护性原则：系统架构应具备良好的模块化特性，便于未来功能扩展与版本迭代，同时保证代码结构清晰、文档完备，便于开发者后期维护、调试与更新。
- (5) 用户友好性原则：系统交互界面与使用流程应尽量简洁明了，降低用户操作门

槛，同时配备完整的用户文档及说明，引导非专业用户也能快速上手与使用。

4.2.2 设计目标

本系统的设计目标是构建一个高效、稳定且易于部署的远控木马早期检测系统，能够在木马运行初始阶段，依托少量初始流量数据精准识别潜在威胁，实现对攻击链条的阻断，最大限度降低安全事件带来的风险与破坏。系统需兼顾高性能与高可用性，具备持续监测、快速分析及及时响应异常流量行为的能力。为提升实用性，检测功能应与自动处置机制紧密结合，实现从检测到终止的完整闭环操作，最大程度减轻用户干预成本。在模型推理方面，系统采用基于 n -gram 与 TF-IDF 的组合特征提取方法，并结合轻量级的随机森林模型实现快速判别，在保证检测精度的同时兼顾实时性与计算效率。通过以上策略，系统能够实现对远控木马攻击行为的早期识别与响应，提升主机对潜在威胁的防御能力，为实际应用环境中的主机安全防护提供一种可行、具备扩展性的技术方案。

4.2.3 系统工作流程

本系统为远控木马早期检测系统，主体功能是对远控木马进行早期检测，防止用户受到远控木马的侵害。用户根据需求开启系统后，系统将实时捕获用户主机中的网络流量，并从中筛选建立连接的会话，采集客户端即用户主机发出的前 k 个有载荷的 TCP 数据包，并从中生成用于检测的流量字节短序列，通过特征计算，输入到训练好的模型中进行检测。当检测结果为远控木马流量时，系统将自动终止对应的木马进程，生成检测报告并发送到指定邮箱。本系统的运行流程如图 4.1 所示。

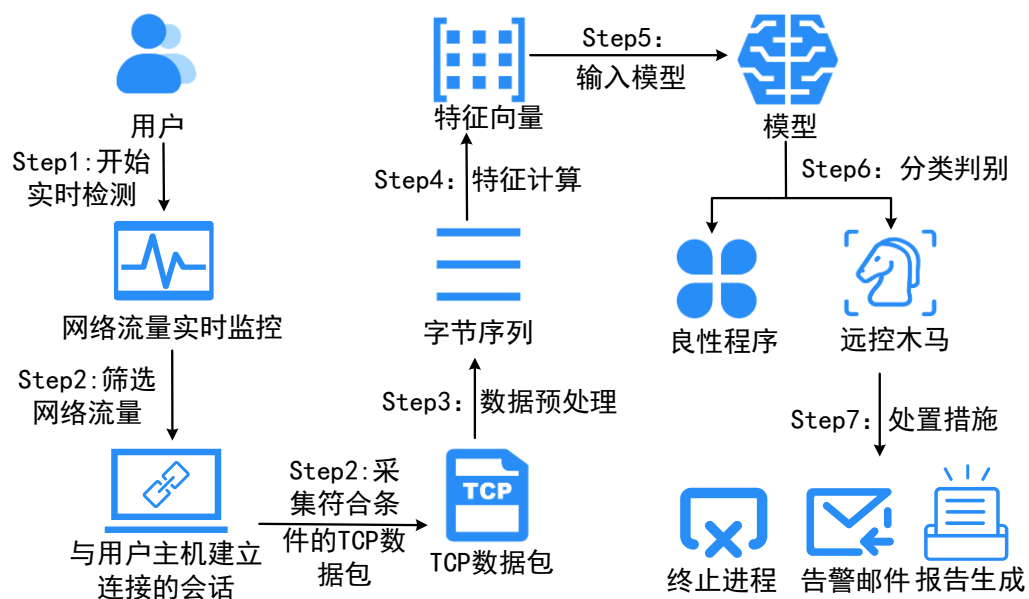


图 4.1 系统的运行流程

4.3 系统功能设计

4.3.1 系统功能模块划分

本系统需要能够在用户主机中实时捕获应用程序产生的网络流量且快速判别所捕获的应用流量是否属于远控木马流量。当检测到远控木马流量时，系统将通过流量的相关信息定位到远控木马的程序进程并实现自动终止，并向用户发送邮箱告警，生成报告保存到本地。

因此，本系统必备的功能分为以下几个模块：实时网络流量捕获模块、远控木马流量检测与自动终止模块、邮箱告警模块、Pcap包离线检测模块和检测报告生成模块。系统总体框架如图 4.2 所示。

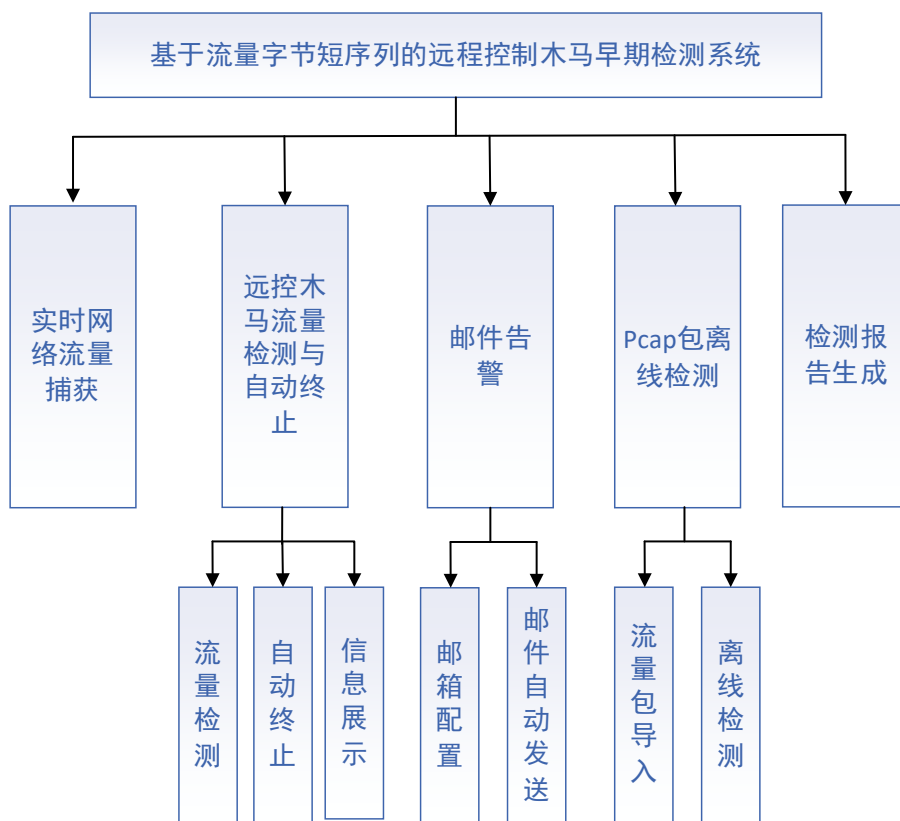


图 4.2 系统总体框架

4.3.2 系统功能模块设计

(1) 实时网络流量捕获模块设计

为满足实时性与可扩展性的要求，本模块设计采用 Python 中 Scapy 库进行底层数据包捕获与处理。用户启动检测功能后，系统会自动进入监听状态，对经过本地网络接口的流量进行捕获与解析，并记录满足条件的数据包信息，以便于后续检测模块进行进

一步分析。

实时网络流量捕获模块的处理流程如图 4.3 所示。

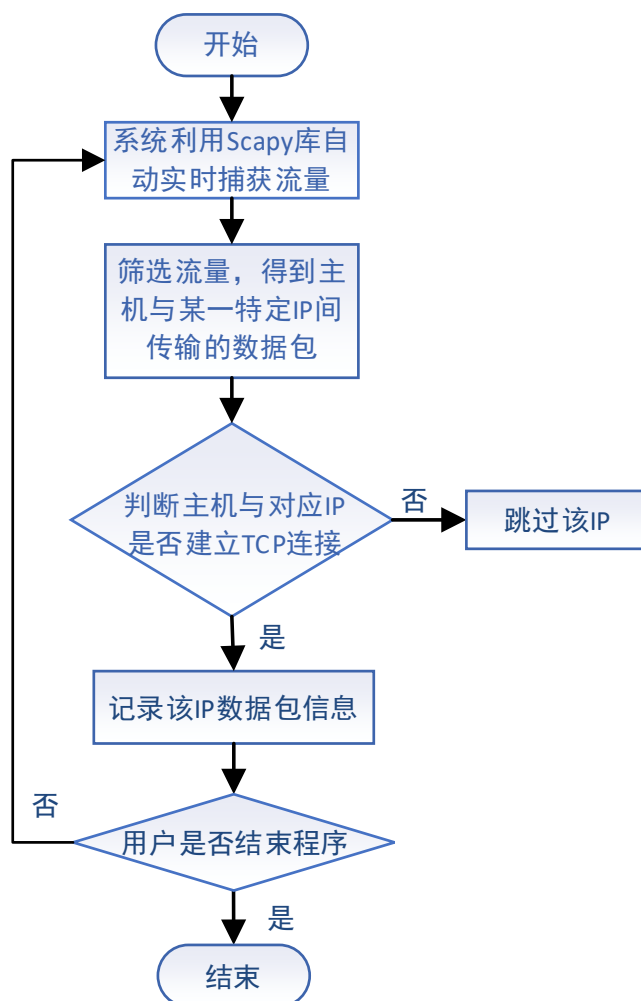


图 4.3 实时网络流量捕获流程图

(2) 远控木马流量检测与自动终止模块设计

在远控木马流量检测与自动终止模块的设计中，系统旨在基于在网络流量捕获阶段捕获到的网络数据包，对潜在的远控木马通信行为进行早期检测，并在检测结果为远控木马流量的情况下，及时采取终止的处置措施以保障主机安全。

模块首先对从实时流量捕获模块中接收到的 TCP 数据包进行预处理，提取其中的有效负载并构造短字节序列，作为检测模型的输入特征。检测模型基于训练好的分类器，对该字节序列进行快速判断，从而识别是否存在远控木马通信行为。一旦检测结果表明该流量为远控木马通信，系统将立即根据通信中的源 IP 信息定位到与该通信行为相关联的本地进程，并调用操作系统提供的 API 接口对该进程及其可能存在的子进程执行强

制终止操作。该机制可有效切断远控连接，防止远控木马持续运行导致的信息泄露、权限控制等安全风险。

远控木马流量检测与自动终止模块的处理流程如图 4.4 所示。

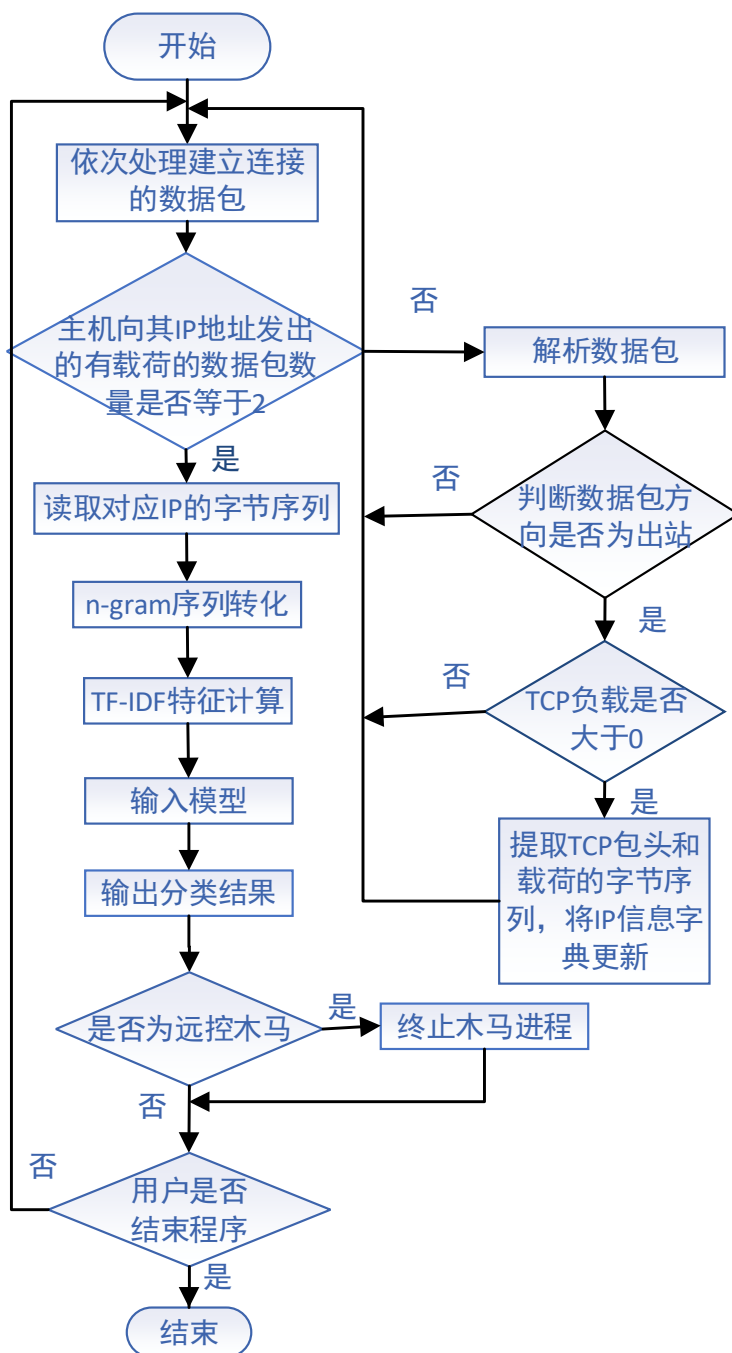


图 4.4 远控木马流量检测与终止流程图

(3) 邮件告警模块设计

在邮件告警模块的设计中，系统需在成功检测到远控木马通信行为后，自动生成并发送结构化的告警邮件，通知用户当前已发生的安全事件及系统处置结果。

邮件告警模块的的处理流程如图 4.5 所示。

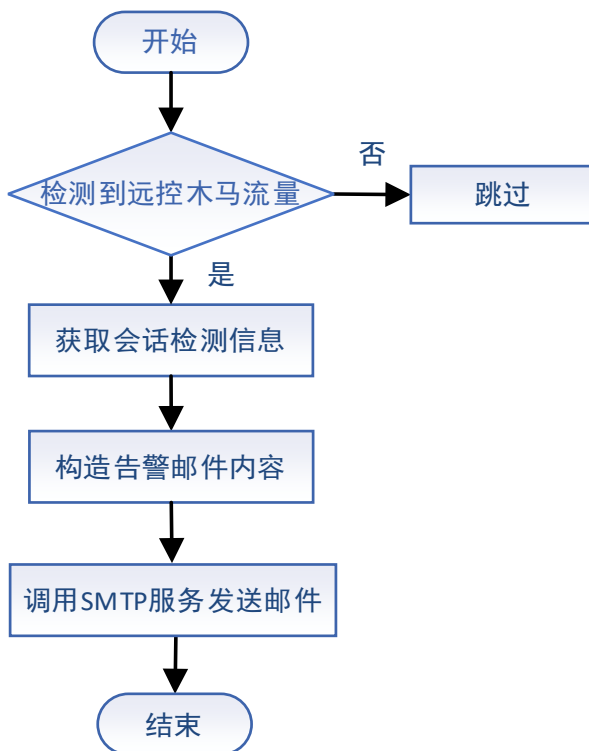


图 4.5 邮件告警流程图

(4) Pcap 包离线检测模块设计

Pcap 包离线检测模块整体流程以数据驱动方式展开。用户通过界面上传 Pcap 格式的网络流量文件，并指定本地主机 IP 作为分析基础。系统在解析 Pcap 文件时，会按时间顺序逐一处理数据包。针对每个与本地主机发生通信的外部 IP 地址，系统动态维护通信会话状态，构建 IP 信息字典对应项，并逐步积累与其相关的数据包内容。在检测逻辑上，模块设置处理触发条件，如通信已建立连接且本地主机已向目标 IP 发送特定数量的有效载荷数据包。满足条件后，系统即对该通信会话提取数据负载内容，并构造字节短序列用于后续检测。特征输入检测模型后，系统将根据模型输出判断该会话是否包含远控木马流量。模块最终对所有会话的检测结果进行聚合分析，统一生成检测简报并保存为报告文件，为用户提供清晰的检测结论与辅助决策依据。

Pcap 包离线检测模块的处理流程如图 4.6 所示。

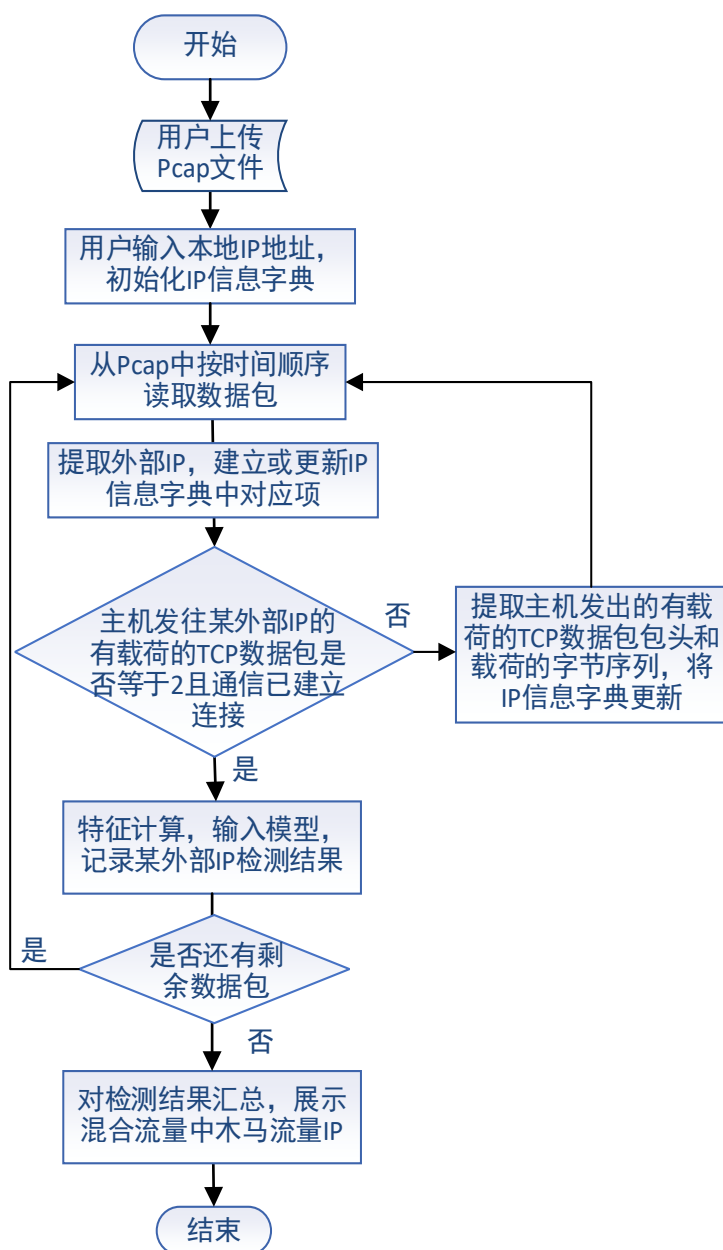


图 4.6 Pcap 包离线检测流程图

(5) 检测报告生成模块设计

检测报告生成模块旨在为系统检测到的远控木马通信行为提供标准化、结构化的结果输出，便于用户对网络攻击行为进行记录、审计与追踪。模块设计注重信息的完整性与可读性，通过自动化流程将检测过程中的关键数据整理为统一格式的报告文件。系统在运行过程中，维护一个存储每一个外部通信 IP 的通信信息的 IP 信息字典，动态存储通信建立时间、字节序列特征等基础信息。当检测模块判定某一通信会话为远控木马流量后，系统将立即补充获取该通信所关联的本地进程信息，包括进程 ID（PID）、进程

名称及进程类型，并执行相应的处置操作。模块通过结构化文本格式将全部信息按统一字段顺序组织，自动构建检测报告。报告文件以“report_年月日_时分秒.txt”为命名规则自动生成，并存储至系统预设的 reports 文件夹下，确保报告具备唯一性、可追溯性和良好的管理性。

检测报告生成模块的处理流程如图 4.7 所示。

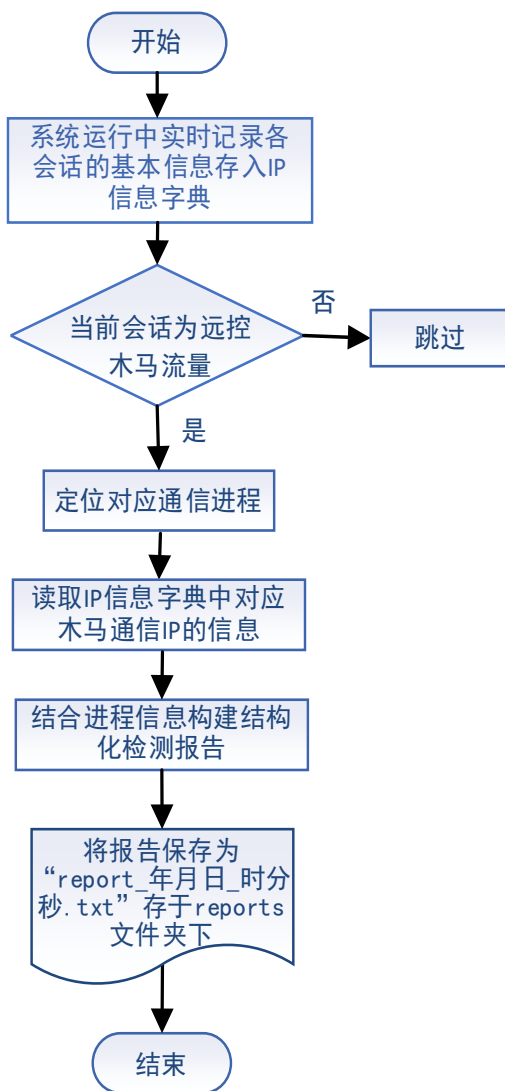


图 4.7 检测报告生成流程图

4.4 本章小结

本章围绕远控木马早期检测系统展开了需求分析与总体设计，明确了系统需具备的关键功能与性能要求，并完成了系统的整体架构设计和功能模块划分以及各个模块的设计，为后续实现提供了依据。



第五章 系统实现

5.1 系统部署方式

系统部署采用本地独立运行的方式，在用户主机上直接安装并执行，无需依赖额外的服务器资源或云端平台。用户只需通过安装包启动系统，即可实现对本地主机网络流量的实时监控与远控木马行为的检测与处置，部署过程简单，便于后期维护与更新。

5.2 实时网络流量捕获模块实现

实时网络流量捕获模块基于 Python 中 Scapy 库的 sniff 函数实现对实时网络流量的监听和捕获。通过指定网络接口和过滤规则，仅捕获 TCP 数据包，并对于每个数据包回调处理函数 packet_handler 进行处理，其中 packet_handler 函数为远控木马流量检测与终止的处理函数，通过该处理逻辑可以实现对于实时网络流量捕获模块与木马流量检测模块的整合衔接。

核心代码如下：

```
from scapy.sendrecv import sniff  
  
def start_sniffing():#启动嗅探  
    sniff(filter="tcp", prn=packet_handler, store=0)
```

5.3 远控木马流量检测与自动终止模块实现

远控木马流量检测与自动终止模块通过实时监听网络流量，提取符合条件的流量会话，并从中提取字节序列，结合特征计算模型计算特征，结合训练好的随机森林模型对流量进行判别，并在检测到远控木马通信后，自动定位并终止相关进程。

整个流程使用 ConnectionState 类来跟踪每个外部 IP 的连接状态以及通信过程中产生的流量字节序列等信息。利用 sequence 属性记录 TCP 握手状态序列，确保出现连续的 SYN->SYN+ACK->ACK 三次握手序列，以确保只处理建立连接的通信。当某一个 IP 的 sequence 序列为[SYN,SYN+ACK,ACK]时，代表其连接被成功建立。一旦检测到三次握手成功，其 connected 属性将被置为 True，系统才会开始记录此连接中的负载用于后续分析。而 byte_sequences 则用于缓存连接建立后所采集到的符合条件的 TCP 数据包中提取的字节序列。data_packet_count 属性用于计数已建立连接的、方向为“出站”的有载荷的 TCP 数据包的数量，当其数量为 2 的时候则触发检测逻辑。具体来说，系统会为每



个目标 IP 维护一个对应的 ConnectionState 实例，利用三次握手标志的顺序变化动态更新 sequence。若发现顺序错误（如握手顺序错乱或不完整），该目标 IP 会被加入跳过列表 skipped_ips，避免后续处理错误数据。只有在 connected 为真，且满足 data_packet_count=2 的条件下，系统才会启动木马检测流程，从而保证检测的准确性与数据的合法性。

```
class ConnectionState:
    def __init__(self):
        self.sequence = [] # 握手状态序列
        self.connected = False # 建立连接状态
        self.byte_sequences = [] # 字节序列
        self.data_packet_count = 0 # 符合条件的数据包数
    connection_states = defaultdict(ConnectionState)
    skipped_ips = set() # 需要跳过的 ip
```

其中，判断建立连接状态的代码如下，通过解析数据包的 flag 标志位来进行判断，如果连续三个数据包的标志位按 SYN->SYN+ACK->ACK 出现，则代表建立连接，其中对于 SYN 可以连续出现，用于表示重连的情况，属于正常，而其余则视为非法建立连接的序列，加入到跳过列表：

```
state = connection_states[target_ip]
if not state.connected: # 三次握手检测
    if flags == "S": # SYN
        if not state.sequence:
            state.sequence.append('SYN') # 将SYN 加入握手序列
        elif state.sequence[-1] == 'SYN':
            pass
        else:
            skipped_ips.add(target_ip) # 非法连接，跳过处理
    return
```

```
elif flags == "SA": # SYN+ACK
    if state.sequence and state.sequence[-1] == 'SYN':
        state.sequence.append('SYN+ACK') # 将SYN+ACK 加入握手序列
    else:
        skipped_ips.add(target_ip) # 非法连接, 跳过处理
        return
elif flags == "A": # ACK
    if state.sequence and state.sequence[-1] == 'SYN+ACK':
        state.sequence.append('ACK') # 将ACK 加入握手序列
        state.connected = True # 成功建立连接
    else:
        skipped_ips.add(target_ip) # 非法连接, 跳过处理
        return
```

对于已经建立连接的 IP，若其方向为出站且具有 TCP 载荷的 TCP 数据包，则 data_packet_count 加 1，并提取其 TCP 数据包的包头与载荷的字节序列。

```
elif state.connected and direction == 'outbound' and
len(tcp_layer.payload) > 0: # 若建立连接且为有载荷的出站TCP 数据包
    state.data_packet_count += 1 # 满足条件的数据包数加1
    tcp_header = (
        tcp_layer.seq.to_bytes(4, 'big') +
        tcp_layer.ack.to_bytes(4, 'big') +
        len(tcp_layer.payload).to_bytes(1, 'big') +
        tcp_layer.flags.to_bytes(1, 'big') +
        tcp_layer.window.to_bytes(2, 'big') +
        tcp_layer.chksum.to_bytes(2, 'big') +
        tcp_layer.urgptr.to_bytes(2, 'big') +
        tcp_layer.options if isinstance(tcp_layer.options,
bytes) else b'') # 获取TCP 数据包包头
```

对于满足条件的数据包数量达到 2 时，使用 `featureGeneration_online()` 函数，将满足检测条件的 IP 信息以及字节序列输入到检测模型进行检测。并将检测完成的 IP 加入跳过列表。

```
full_data = tcp_header + bytes(tcp_layer.payload) # 包头和载荷
ascii_data = extract_ascii(full_data)
state.byte_sequences.append(ascii_data)
if state.data_packet_count == 2:
    print(f"{target_ip}: 收集完毕，开始检测")
    featureGeneration_online(state.byte_sequences, target_ip,
tcp_layer.sport)
    skipped_ips.add(target_ip) # 防止重复处理
```

`featureGeneration_online()` 函数完成对于字节序列的检测，使用事先训练好的模型进行特征计算以及检测。

```
def featureGeneration_online(byte_sequences, ip, port):
    merged_sequence = ' '.join(byte_sequences)
    byte_sequence = merged_sequence.strip()
    byte_vector = vectorizer.transform([byte_sequence]) # 转为向量
    predict_label = classifier.predict(byte_vector) # 模型预测
    label = "Trojan"
    result_list["type"] = label
    if predict_label[0] == 0: # 获得检测结果
        label = "Benign"
        result_list["type"] = label
```

若检测为木马流量则并结合传入的 IP 信息，结合 `psutil` 提供的 `net_connections()` 接口，通过端口号反查本地通信对应的进程 PID，并调用 `os.kill(pid, signal.SIGTERM)` 终止可疑进程。

```
def get_pid_using_port(port): # 定位进程PID
    try:
        for conn in psutil.net_connections(kind='inet'):
            if conn.laddr and conn.laddr.port == port:
                if conn.pid is not None:
                    return conn.pid
        return None
    except Exception as e:
        print(f"Error occurred: {e}")
        return None

def kill_process(pid): # 终止可疑进程
    try:
        os.kill(pid, signal.SIGTERM) # 尝试使用SIGTERM 信号
        time.sleep(1)
    except OSError as e:
        pass
```

5.4 邮件告警模块实现

该模块在检测到木马通信后调用，用于向用户发送邮件通知。使用 `smtplib` 和 `email` 库构造并发送邮件，内容包括被检测 IP、时间、检测结果等。该模块基于 SMTP 协议实现邮件发送功能，支持主流邮箱平台（如 QQ 邮箱、163 邮箱等），用户可通过配置文件预设收件人邮箱地址。邮件内容自动生成，包含以下关键信息：

- (1) 远控木马服务端 IP 地址：标识木马通信目标，可用于溯源分析。
- (2) 通信建立时间：记录远控连接初次建立的时间点，便于审计。
- (3) 可疑进程 ID 和进程名称：指示与可疑流量相关联的本地进程，便于用户进一步排查或确认。
- (4) 处置结果：显示系统是否已成功终止该进程。

```
import smtplib
from email.mime.text import MIMEText
from email.header import Header
def email_main_fun(subject, body): # 构建邮件
    msg = MIMEText(body, 'plain', 'utf-8')
    msg['Subject'] = Header(subject, 'utf-8')
    msg['From'] = manager_mail # 管理员配置的发件邮箱地址
    msg['To'] = user_mail # 用户配置的收件人邮箱地址
    smtp_server = 'smtp.qq.com'
    smtp_port = 587
    sender_email = manager_mail
    password = manager_code # 管理员的邮箱授权码
    try:
        with smtplib.SMTP(smtp_server, smtp_port) as server:
            server.starttls()
            server.login(sender_email, password)
            server.sendmail(sender_email, [msg['To']],
msg.as_string())
    except smtplib.SMTPException as e:
        pass
```

5.5 Pcap 包离线检测模块实现

该模块使用 dpkt 库对离线 Pcap 文件进行解析和逐包处理。核心流程包括：TCP 三次握手状态跟踪、目标 IP 识别、方向判断、字节序列提取与模型调用。通过状态对象记录每个目标 IP 的连接情况和数据包内容，并在捕获 2 个数据包后进行特征分析。最终将结果保存至本地临时文件。对于 TCP 三次握手状态跟踪、目标 IP 识别、方向判断、字节序列提取与模型调用的核心逻辑均与实时检测逻辑类似。

5.6 检测报告生成模块实现

当某个通信对被检测为远控木马时，该模块将所有数据组织为结构化字典。模块调

用文本模板生成检测报告，内容包括木马 IP、时间戳、进程 ID、进程名、处置状态等字段。报告内容采用统一格式和字段顺序编写，并通过 `datetime` 模块生成唯一的时间戳文件名，确保文件不会重名。最终报告通过 Python 的 `open()` 函数写入系统的 `reports/` 目录下，供用户后续查看和归档。

```
report_dir = "reports"
os.makedirs(report_dir, exist_ok=True)
report_filename =
f"report_{ending_time.strftime('%Y%m%d_%H%M%S')}.txt"
report_path = os.path.join(report_dir, report_filename)
report_content = (
    f"==== RATMS 检测报告 =====\n"
    f"远控木马通信 IP: {ip}\n"
    f"检测开始时间: {start_format_time}\n"
    f"检测结束时间: {ending_format_time}\n"
    f"检测耗时（秒）: {time_diff:.2f}\n"
    f"本地进程 PID: {pid}\n"
    f"进程名称: {pname}\n"
    f"进程类型: {process_type}\n"
    f"处置状态: {state}\n"
    f"=====\n"
)
```

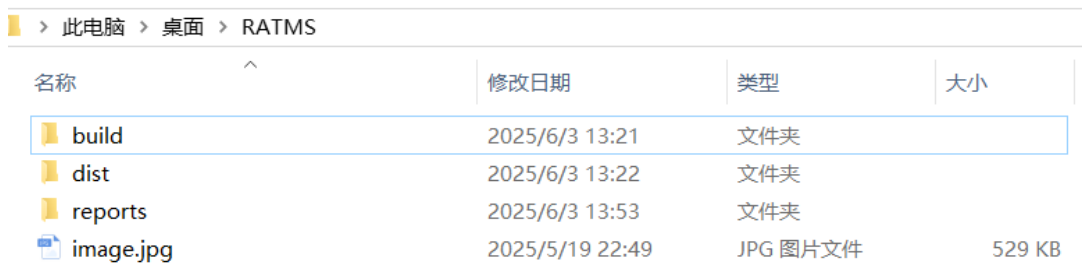
5.7 本章小结

本章主要介绍了远控木马早期检测系统的各个模块的具体实现过程，包括系统部署方式、实时流量捕获、远控木马检测与自动终止、邮箱告警、Pcap 包离线检测以及检测报告生成等功能模块。通过对各模块的实现进行详细说明，展示了系统如何在实际运行中完成对远控木马流量的检测与处置。

第六章 系统测试

6.1 系统测试环境

基于流量字节短序列的远程控制木马早期检测系统基于 PySide2 开发框架实现，在经过运行项目代码调试和测试完成后，使用 PyInstaller 命令打包项目以生成可执行的系统程序。测试主机无须运行项目代码和配置代码环境，直接运行可执行程序即可使用系统。在项目打包完成后，需将如图 6.1 中的项目文件拷贝到测试主机上，测试主机直接运行如图 6.2 中所示项目文件中 dist 目录中 RATMS 下的可执行程序 RATMS.exe 即可使用本系统。



> 此电脑 > 桌面 > RATMS			
名称	修改日期	类型	大小
build	2025/6/3 13:21	文件夹	
dist	2025/6/3 13:22	文件夹	
reports	2025/6/3 13:53	文件夹	
image.jpg	2025/5/19 22:49	JPG 图片文件	529 KB

图 6.1 项目文件夹



> 此电脑 > 桌面 > RATMS > dist > RATMS >		
名称	修改日期	类型
_internal	2025/6/3 13:33	文件夹
RATMS.exe	2025/6/3 13:22	应用程序

图 6.2 可执行文件

本系统所使用的测试设备的具体配置如表 6.1 所示。

表 6.1 测试设备的具体配置

开发环境/开发技术	具体描述
处理器	Intel(R) Core(TM) i7-11800H @ 2.30GHz 2.30 GHz
机带 RAM（内存）	16GB
操作系统	Windows 10 专业版_22H2（64 位）
可执行程序	RATMS.exe
依赖环境	reports、saved_model、build、dist



6.2 功能测试

6.2.1 实时网络流量捕获

用户开启检测功能后，系统自动调用底层数据包捕获接口开始实时收集网络流量。为了验证模块的稳定性与准确性，测试步骤包括：在测试主机中运行本系统，启用检测功能，并保持系统在联网状态下持续运行 5 分钟。期间，通过浏览网页、访问服务器等方式生成正常网络活动，同时使用第三方工具 Wireshark 对网络接口进行对比抓包，以验证系统捕获与筛选的准确性。系统在测试运行期间表现稳定，能够持续捕获网络流量数据包，并准确识别出 TCP 协议数据、对数据包按目标 IP 进行分类，且能成功判断连接是否建立。系统运行过程中未出现崩溃、卡顿或明显丢包现象，测试输出如图 6.3 所示，具体的功能测试结果整理见表 6.2。

表 6.2 实时网络流量捕获测试用例

测试项目	测试内容	执行步骤	预期结果	分析结果
网络流量捕获	系统能否自动调用流量捕获接口并成功接收数据包	1. 启动系统并点击“开启检测”按钮 2. 运行程序或浏览网页，制造正常网络活动 3. 观察系统是否开始自动抓取流量数据包	系统自动调用捕获流量接口，得到网络流量数据包并发送给远控木马检测模块	符合预期
数据包筛选	系统能否正确筛选出 TCP 协议类型的数据包并根据 IP 分类处理	1. 启动系统并开启检测 2. 打开多个网页，访问不同域名 3. 检查抓包结果是否根据不同 IP 进行分组处理	能够正确地筛选出 TCP 数据包，且能够根据 IP 对流量数据包进行分类处理	符合预期
连接建立状态识别	是否能判断 TCP 流量是否完成三次握手，确认连接是否已建立	1. 启动系统并开启检测 2. 打开多个网页，访问不同域名 3. 检查系统是否分析 SYN、SYN-ACK、ACK 报文以判断连接状态	系统正确识别三次握手，能判断连接是否建立	符合预期
流量传输稳定性测试	系统长时间运行下，是否能持续稳定捕获流量并不丢包	1. 启动系统检测功能 2. 进行网页浏览等活动持续 5 分钟 3. 查看系统捕获流量是否中断、异常或丢包	系统稳定运行，无中断、异常日志或流量丢失；捕获流量数据持续更新	符合预期

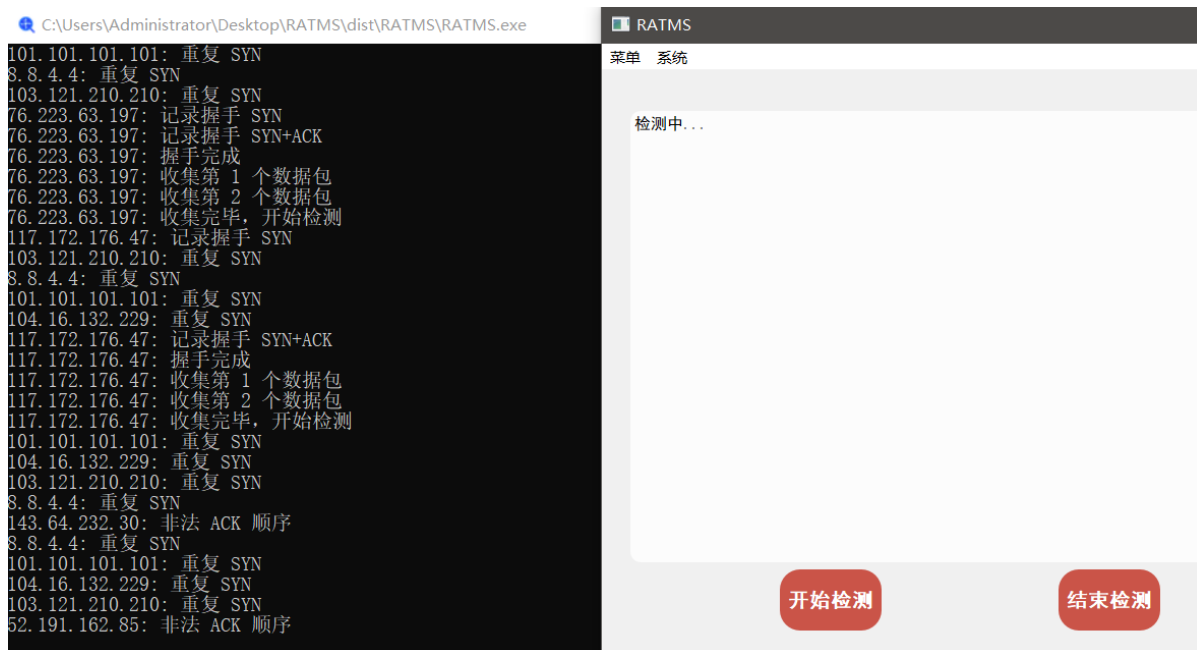


图 6.3 实时网络流量捕获测试

6.2.2 远控木马流量检测与自动终止

本文模拟测试了现实场景中远控木马的检测与自动终止，用户点击开启检测按钮即可开始进行网络流量检测，若在检测过程中出现远控木马流量，系统会回显远控木马的源 IP 地址、木马的进程 ID 与进程名以及处理状态。本文选用不在方法训练集中的知名远控木马软件 GhostDZ RAT 远控木马软件。本次测试将测试环境分为控制端（安装控制端组件的云服务器）和被控端（测试主机），以对应现实中的远控木马攻击场景。

如图 6.4 所示，首先将远控木马控制端运行在 IP 为 49.232.228.35 的云服务器中，并生成名为 GhostDZ RAT.exe 的远控木马客户端，将其拷贝到测试主机。

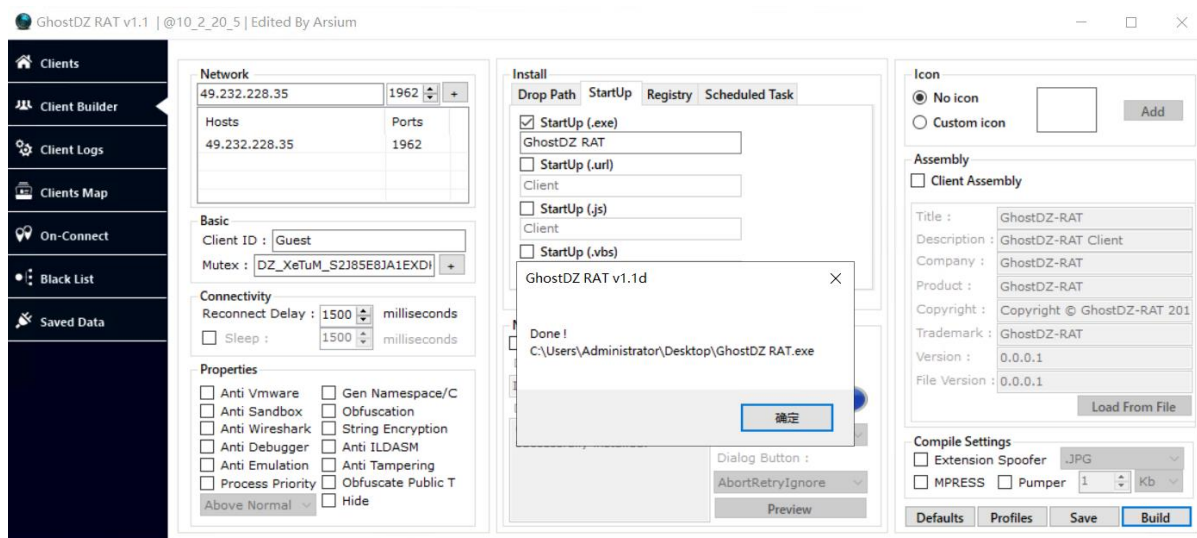


图 6.4 远控木马客户端生成

在测试主机中运行检测程序后，程序将对主机中流量进行实时监听，并依次处理，此时在测试主机中运行远控木马客户端，从任务管理器中可以观察到对应出现了进程号为 21516 的 GhostDZ RAT.exe 的进程信息，如图 6.5 所示。

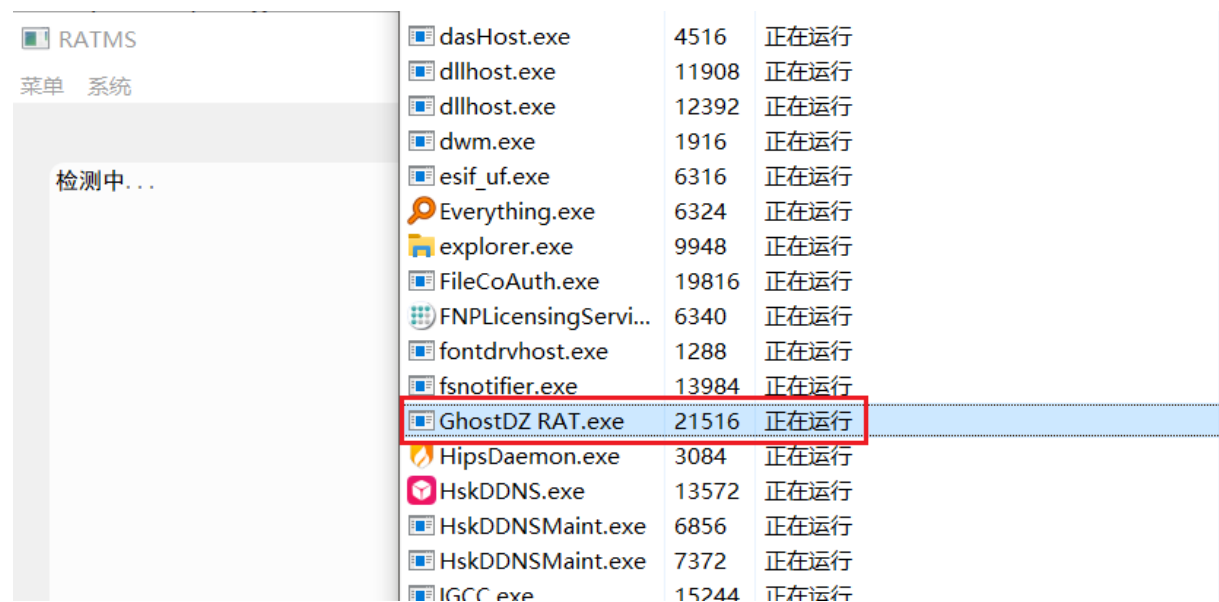


图 6.5 远控木马开启运行

运行约 3s 左右，检测程序检测到远控木马流量，并对其进行终止，对应进程被结束，在任务管理器中消失，同时检测告警信息回显在系统页面中，检测到木马通信 IP，即控制端 IP，与运行远控木马控制端的云服务器 IP 一致，并同时回显了远控木马对应系统进程以及进程名，如图 6.6 所示。

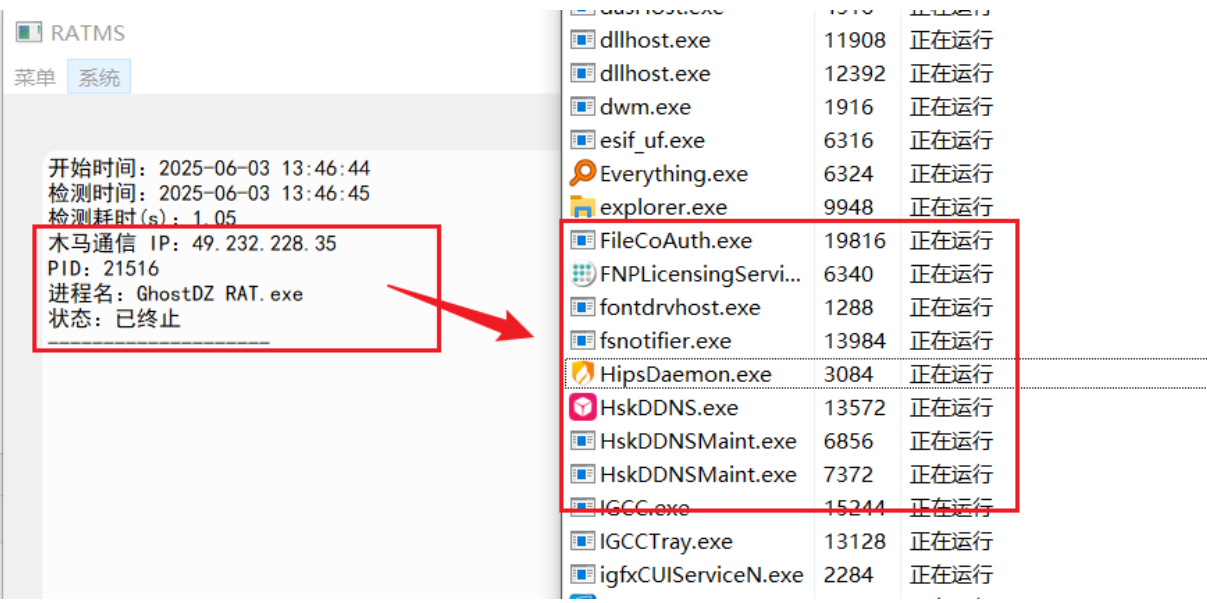


图 6.6 远控木马流量检测与终止

总体测试结果如表 6.3 所示，对于远控木马流量检测与终止模块性能良好，符合预期：

表 6.3 远控木马检测与终止测试结果

测试数据（样本名称）	样本属性	预期结果	测试结果	是否自动终止
GhostDZ RAT	Trojan	Trojan	Trojan	是

6.2.3 邮件告警

如图 6.7 所示，系统在检测到远控木马攻击时，通过系统中配置的用户邮箱地址向其发送告警文件，文件包含详细的告警信息。即使用户不在系统前，也能保持对系统安全的关注，并且可以根据邮件中提供的建议措施采取必要的行动，加强对应的安全防护。



图 6.7 告警邮件

6.2.4 Pcap 包离线检测

用于 Pcap 包离线检测的 Pcap 包为在 IP 为 192.168.52.192 的测试主机抓取的混合流量，主机上同时运行着良性程序和远控木马客户端（控制端运行在 IP 为 49.232.228.35 的云服务器），用于测试系统对混合流量中良性程序流量和木马流量的识别效果。首先，运行系统，选择离线检测模块，上传用于检测的 Pcap 文件，如图 6.8。

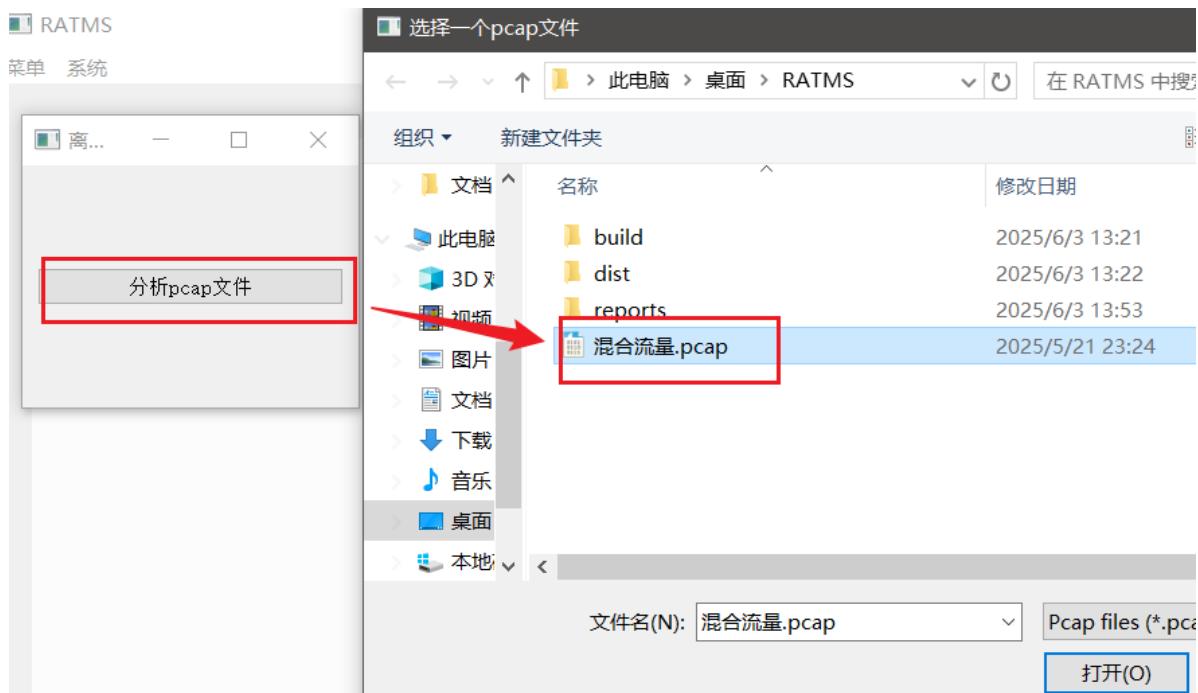


图 6.8 Pcap 流量包上传

随后输入该流量包中的主机 IP，如图 6.9，随后开始检测，系统将提取流量包中所有与主机 IP 通信的 IP，并分析其对应的数据包，给出检测结果，如图 6.10，系统的离线检测模块可成功检测出混合 Pcap 流量包中的木马流量 IP。

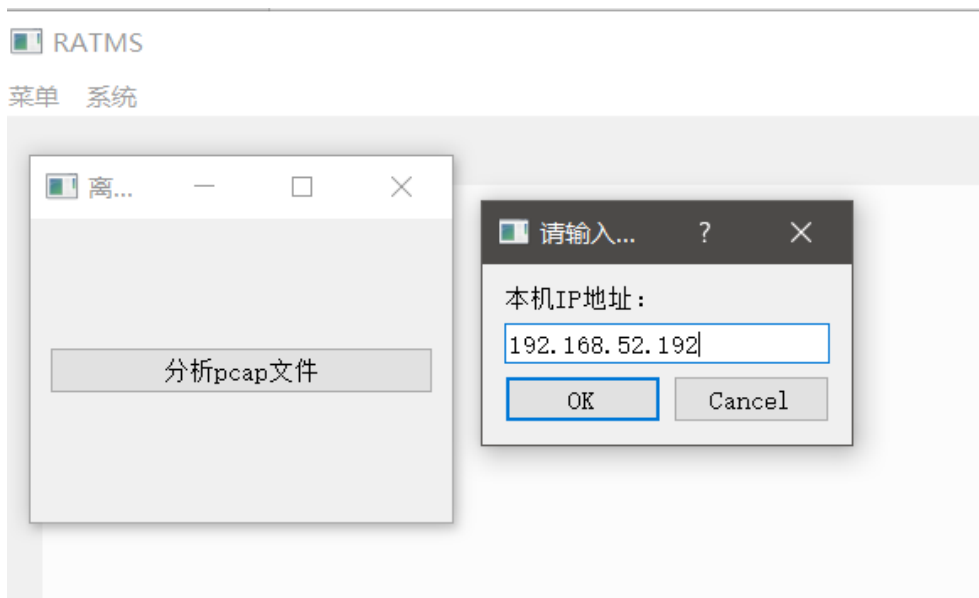


图 6.9 主机 IP 输入

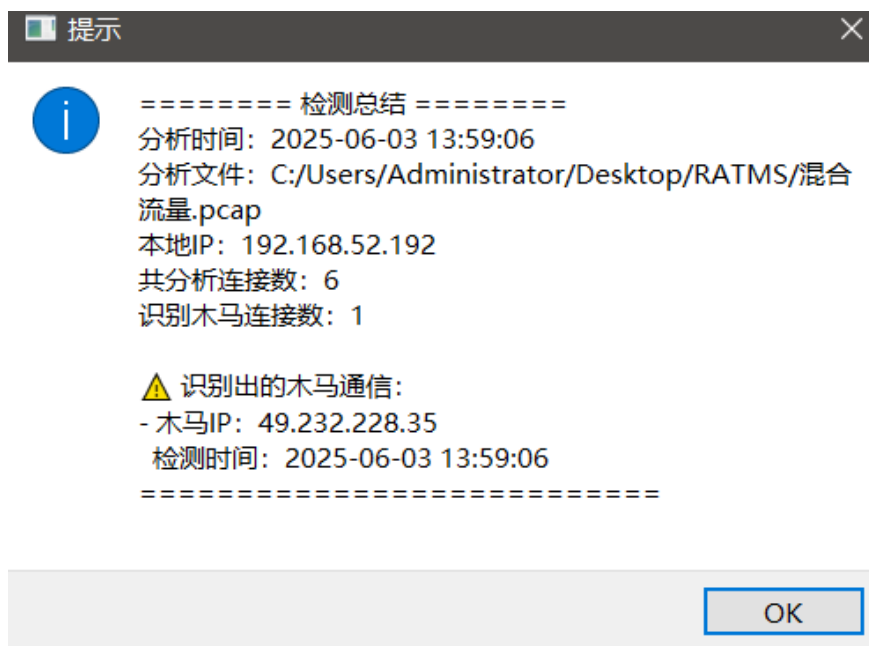


图 6.10 离线检测结果

6.2.5 检测报告生成

系统完成检测后会自动生成检测报告，生成检测报告模块的测试用例选用 GhostDZ RAT 远控木马软件，如图 6.11 所示，系统完成检测后，用户根据系统中检测报告页面所给出文件目录可以看检测报告，得到检测到的远控木马攻击对应的基本信息。



图 6.11 检测报告



6.3 测试总结

测试总结如表 6.4 所示，系统各界面均显示正常，各功能都符合预期。

表 6.4 测试结果

测试	测试描述	执行步骤	预期结果	实际结果
界面测试	各个功能界面运行情况	系统运行，点击按钮进入各界面	正常显示	符合预期
	实时网络流量捕获	点击开始检测按钮	能够捕获网络流量，并且筛选流量数据包	符合预期
	远控木马检测与自动终止	点击开始检测按钮	能够分类良性程序流量和远控木马流量并自动终止对应进程	符合预期
功能测试	邮件告警	点击开始检测按钮，等待检测结果	能够及时且准确发送告警邮件	符合预期
	Pcap 包离线检测	点击离线检测，上传 Pcap 文件，输入主机 IP	能够读入 Pcap 文件，识别其中 IP 对应的流量类型	符合预期
	检测报告生成	点击开始检测按钮，检测完成自动生成	能够生成远控木马攻击检测报告	符合预期

6.4 本章小结

本章对系统的主要功能模块和性能进行了测试，验证了系统在实际运行中的稳定性和准确性。测试结果表明，各功能模块均能按预期工作，系统具备良好的检测能力与实用性。



第七章 总结与展望

7.1 总结

实现对远控木马流量的高效检测一直是网络安全研究中的重要课题。当前主流的检测方法多依赖深度学习或统计特征分析，尽管在准确率方面取得了一定成果，但在实际应用中仍面临检测延迟高、资源消耗大、泛化能力有限等问题。为此，本文设计一种基于流量字节短序列的远控木马早期检测方法 RATMS，旨在在木马运行的初始阶段，通过更少的数据实现对其行为的有效识别，从而提升检测的及时性和适用性。

该方法以应用程序运行初始阶段客户端建立连接后发送的前 k 个有载荷的 TCP 数据包为基础，提取其包头及载荷的字节序列作为输入数据，采用 n -gram 结合 TF-IDF 进行特征表示，随后通过随机森林算法实现分类判别。相较于传统方法，本方案在减少数据截断信息损失的同时，保留了较为完整的通信特征。实验部分选取 45 个良性程序与 45 个远控木马的网络流量样本进行评估，结果显示，该方法仅使用从客户端首次建立连接后发送的前两个具有有效载荷的 TCP 数据包，对未知远控木马流量的检测准确率可达 97.3%，具有较强的泛化能力与检测及时性。

在系统实现方面，本文基于 PySide2 框架设计并开发了一套远控木马早期检测系统。该系统具备实时流量采集、自动化分析与告警、可疑进程自动终止等功能，能够在用户主机中持续运行并对应用程序产生的网络通信流量进行监测与识别。当系统检测到异常流量特征时，将自动定位到相关进程并终止，避免进一步的信息泄露风险，并在处置后向用户发送邮件。

总体来看，本文从方法设计到系统实现，兼顾了检测效率、准确性与实用性，为远控木马的早期发现与防御提供了一种轻量、高效且可部署的解决方案。

7.2 展望

综合来看，本文设计的方法在准确性、检测及时性与实现复杂度之间取得了较为平衡的效果，适用于对远控木马攻击进行早期防范的场景。然而，本文仍存在一定局限性，如当前系统的检测报告功能较为简略，且尚未在复杂企业级网络环境中进行部署与评估。后续工作将进一步拓展该系统在多样化场景下的适应能力，提升其实用性与可解释性，并在真实环境中进行大规模测试，以验证方法的稳定性和扩展性。



参考文献

- [1] 国家互联网应急中心.2021 年上半年我国互联网网络安全监测数据分析报告 [DB/OL].<https://www.cert.org.cn/publish/main/upload/File/first-half%20%20year%20cybersecurity%20report%202021.pdf>
- [2] 中国网络空间安全协会.2023 年网络安全态势研判分析年度综合报告（2024 第 2 期） [DB/OL].<https://www.cybersac.cn/detail/1760575360354787330>
- [3] 瑞星.2024 年中国网络安全报[DB/OL].<http://www.rising.com.cn/2025/baogao/>
- [4] Gaudesi M, Marcelli A, Sanchez E, et al. Challenging anti-virus through evolutionary malware obfuscation[C]//Applications of Evolutionary Computation: 19th European Conference, EvoApplications 2016, Porto, Portugal, March 30--April 1, 2016, Proceedings, Part II 19. Springer International Publishing, 2016: 149-162.
- [5] Aghakhani H, Gritti F, Mecca F, et al. When malware is packin'heat; limits of machine learning classifiers based on static analysis features[C]//Network and Distributed System Security Symposium. Internet Society, 2020: 1-21.
- [6] Gajrani J, Sarswat J, Tripathi M, et al. A robust dynamic analysis system preventing SandBox detection by Android malware[C]//Proceedings of the 8th International Conference on Security of Information and Networks. 2015: 290-295.
- [7] Nappa A, Papadopoulos P, Varvello M, et al. Pow-how: An enduring timing side-channel to evade online malware sandboxes[C]//Computer Security—ESORICS 2021: 26th European Symposium on Research in Computer Security, Darmstadt, Germany, October 4–8, 2021, Proceedings, Part I 26. Springer International Publishing, 2021: 86-109.
- [8] Caviglione L, Choraś M, Corona I, et al. Tight arms race: Overview of current malware threats and trends in their detection[J]. IEEE Access, 2020, 9: 5371-5396.
- [9] Afianian A, Niksefat S, Sadeghiyan B, et al. Malware dynamic analysis evasion techniques: A survey[J]. ACM Computing Surveys (CSUR), 2019, 52(6): 1-28.
- [10] 陈铁明, 金成强, 吕明琪, 等. 基于样本增强的网络恶意流量智能检测方法[J]. 通信学报, 2020, 41(6): 128-138.
- [11] Rosenberg I, Shabtai A, Rokach L, et al. Generic black-box end-to-end attack against state of the art API call based malware classifiers[C]//Research in Attacks, Intrusions, and Defenses: 21st International Symposium, RAID 2018, Heraklion, Crete, Greece, September 10-12, 2018, Proceedings 21. Springer International Publishing, 2018: 490-510.
- [12] Chen X, Li C, Wang D, et al. Android HIV: A study of repackaging malware for evading machine-learning detection[J]. IEEE Transactions on Information Forensics and Security, 2019, 15: 987-1001.
- [13] Maiorca D, Ariu D, Corona I, et al. Stealth attacks: An extended insight into the obfuscation effects on android malware[J]. Computers & Security, 2015, 51: 16-31.
- [14] Cheng B, Ming J, Fu J, et al. Towards paving the way for large-scale windows malware analysis: Generic binary unpacking with orders-of-magnitude performance boost[C]//Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security. 2018: 395-411.
- [15] Pék G, Bencsáth B, Buttyán L. nEther: In-guest Detection of Out-of-the-guest Malware Analyzers[C]//Proceedings of the fourth European workshop on system security. 2011: 1-6.
- [16] Yokoyama A, Ishii K, Tanabe R, et al. Sandprint: Fingerprinting malware sandboxes to provide intelligence for sandbox evasion[C]//Research in Attacks, Intrusions, and Defenses: 19th International Symposium, RAID 2016, Paris, France, September 19-21, 2016, Proceedings 19. Springer International Publishing, 2016: 165-187.
- [17] Kakisim A G, Gulmez S, Sogukpinar I. Sequential opcode embedding-based malware detection



- method[J]. Computers & Electrical Engineering, 2022, 98: 107703.
- [18] 孙世淼,刘亚姝,严寒冰.基于多字节频率域可视化和深度学习的恶意软件检测[J].计算机工程与设计,2024,45(08):2272-2280.DOI:10.16208/j.issn1000-7024.2024.08.005.
- [19] Canali D, Lanzi A, Balzarotti D, et al. A quantitative study of accuracy in system call-based malware detection[C]//Proceedings of the 2012 International Symposium on Software Testing and Analysis. 2012: 122-132.
- [20] Farinholt B, Rezaeirad M, Pearce P, et al. To catch a ratter: Monitoring the behavior of amateur darkcomet rat operators in the wild[C]//2017 IEEE symposium on Security and Privacy (S P). Ieee, 2017: 770-787.
- [21] BehradFar M M, HaddadPajouh H, Dehghantanha A, et al. RAT hunter: Building robust models for detecting remote access trojans based on optimum hybrid features[J]. Handbook of Big Data Privacy, 2020: 371-383.
- [22] Lindorfer M, Kolbitsch C, Milani Comparetti P. Detecting environment-sensitive malware[C]//Recent Advances in Intrusion Detection: 14th International Symposium, RAID 2011, Menlo Park, CA, USA, September 20-21, 2011. Proceedings 14. Springer Berlin Heidelberg, 2011: 338-357.
- [23] Yin K S, Khine M A. Network behavioral features for detecting remote access Trojans in the early stage[C]//Proceedings of the 2017 VI International Conference on Network, Communication and Computing. 2017: 92-96.
- [24] 宋紫华,郭春,蒋朝惠.一种基于网络流量分析的快速木马检测方法[J].计算机与现代化,2019,(06):9-15.
- [25] Guo C, Song Z, Ping Y, et al. Prstd: A phased remote access trojan detection method with double-sided features[J]. Electronics, 2020, 9(11): 1894.
- [26] 王晨,郭春,申国伟,等.利用序列分析的远控木马早期检测方法研究[J].计算机科学与探索,2021,15(12):2315-2326.
- [27] Jiang D, Omote K. An approach to detect remote access trojan in the early stage of communication[C]//2015 IEEE 29th international conference on advanced information networking and applications. IEEE, 2015: 706-713.
- [28] Jiang W, Wu X, Cui X, et al. A highly efficient remote access trojan detection method[J]. International Journal of Digital Crime and Forensics (IJDCF), 2019, 11(4): 1-13.
- [29] Wang W, Zhu M, Zeng X, et al. Malware traffic classification using convolutional neural network for representation learning[C]//2017 International conference on information networking (ICOIN). IEEE, 2017: 712-717.
- [30] Chen J, Yin S, Cai S, et al. An efficient network intrusion detection model based on temporal convolutional networks[C]//2021 IEEE 21st International Conference on Software Quality, Reliability and Security (QRS). IEEE, 2021: 768-775.
- [31] 吴丰源,刘明,尹小康,等.基于融合序列的远控木马流量检测模型[J].计算机科学,2024,51(06):434-442.
- [32] 谷勇浩,黄博琪,王继刚,等.基于半监督深度学习的木马流量检测方法[J].计算机研究与发展,2022,59(06):1329-1342.
- [33] Pi B, Guo C, Cui Y, et al. Remote access trojan traffic early detection method based on Markov matrices and deep learning[J]. Computers & Security, 2024, 137: 103628.



致谢

四年的大学时光转瞬即逝，当我写下这篇致谢时，才真正意识到，这段旅程真的要画上句号了。回望过去的点点滴滴，心中充满了感慨。一路走来，每一个给予我帮助和鼓励的人，都让我心怀感激。

首先，特别感谢我的毕业设计指导老师同时也是我大学的导师郭春老师。郭老师总是耐心地指出我在学习和研究中的问题，帮我梳理思路，引导我不断深入思考。这两年来，在老师的指导下参加比赛、做项目、在实验室学习实践，我收获了很多。从最初的迷茫到如今能独立分析问题、调研、提出解决方案，我能真切地感受到自己的成长。更重要的是，老师不仅教会了我知识，也让我明白了面对问题的正确心态。在面对问题时，以前的我总会有畏难情绪，犹豫不决，倾向于把问题想得很难，但老师总是会先让我去试一下。好像也是一种人生哲理，动手去做才能解决问题，只是犹豫而不出发，不会得到答案。

也感谢那些陪我一起熬夜写代码、并肩走过大学时光的朋友们。我们一起经历过无数次被 Bug 折磨的深夜，也一起分享项目完成后的喜悦。有你们在，大学不仅是求学的旅程，更是一段值得反复回味的珍贵回忆。感谢我的家人，一直站在我背后，无条件地相信我、支持我做的每个决定。

最后，也想真诚地感谢一下自己。感谢那个无数次想放弃却依然咬牙坚持的自己，感谢那个不断打碎又重建的自己。希望未来的我能成为一个真正有韧性的人。不破不立，继续出发。